


Technical University of Crete

School of Electrical & Computer Engineering


Diploma Thesis

# Investigating the Applicability of Federated Dynamic Averaging in Federated Low-Rank Adaptation Architectures


**Author:**

Asterinos Karalis

**Thesis Committee:**

Assoc. Prof. Vasilis Samoladas (Supervisor)

Prof. Antonios Deligiannakis

Prof. Minos Garofalakis


A thesis submitted in partial fulfillment of the requirements
for the degree of Diploma in Electrical and Computer Engineering

November 2026

# Abstract

Federated Learning has established itself as a foundational paradigm for privacy-preserving collaborative model training across decentralized data sources. However, scaling modern foundation models to billions of parameters exposes a critical dual bottleneck in federated optimization: an excessive *spatial* communication overhead from transmitting high-dimensional parameter updates, and an inefficient *temporal* communication pattern imposed by rigid, fixed-interval synchronization schedules. Parameter-efficient fine-tuning methods such as Low-Rank Adaptation address the spatial dimension by decomposing updates into compact low-rank factors. Yet, naively applying Low-Rank Adaptation in federated environments introduces a fundamental structural inconsistency: independent averaging of constituent low-rank matrices fails to reconstruct the true global update due to the non-commutativity of matrix multiplication. Furthermore, existing federated Low-Rank Adaptation variants remain temporally inefficient by relying on static synchronization frequencies agnostic to local training dynamics.

To address both dimensions simultaneously, this thesis investigates combining Federated Dynamic Averaging with federated low-rank adaptation. Specifically, we develop and formulate FDA-RoLoRA, an algorithmic formulation that pairs dynamic, variance-driven averaging with Robust Low-Rank Adaptation. This method eliminates the bilinear aggregation inconsistency through alternating updates between down-projection and up-projection matrices, while dynamically regulating synchronization frequency via decoupled, dynamically updated per-matrix variance thresholds. We evaluate the proposed framework on the General Language Understanding Evaluation (GLUE) benchmark using the `roberta-base` model.

# Περίληψη

Η Ομοσπονδιακή Μάθηση έχει καθιερωθεί ως ένα θεμελιώδες πρότυπο για τη συνεργατική εκπαίδευση μοντέλων μηχανικής μάθησης με διατήρηση της ιδιωτικότητας σε αποκεντρωμένες πηγές δεδομένων. Ωστόσο, η κλιμάκωση των σύγχρονων θεμελιωδών μοντέλων (foundation models) σε δισεκατομμύρια παραμέτρους αναδεικνύει μια κρίσιμη διπλή συμφόρηση επικοινωνίας στη διαδικασία της ομοσπονδιακής βελτιστοποίησης: ένα υπέρμετρο *χωρικό* κόστος επικοινωνίας λόγω της μετάδοσης διανυσμάτων παραμέτρων υψηλής διάστασης, και ένα αναποτελεσματικό *χρονικό* μοτίβο επικοινωνίας που επιβάλλεται από άκαμπτα, στατικά χρονοδιαγράμματα συγχρονισμού καθορισμένων διαστημάτων. Οι μέθοδοι προσαρμογής παραμετρικής αποδοτικότητας, όπως η Προσαρμογή Χαμηλής Τάξης (LoRA), αντιμετωπίζουν τη χωρική διάσταση αποσυνθέτοντας τις ενημερώσεις σε συμπυκνωμένους παράγοντες χαμηλής τάξης. Εντούτοις, η αφελής εφαρμογή της Προσαρμογής Χαμηλής Τάξης σε ομοσπονδιακά περιβάλλοντα εισάγει μια θεμελιώδη δομική ανακολουθία: η ανεξάρτητη στάθμιση των επιμέρους πινάκων χαμηλής τάξης αδυνατεί να ανακατασκευάσει την πραγματική καθολική ενημέρωση λόγω της μη αντιμεταθετικότητας του πολλαπλασιασμού πινάκων. Επιπλέον, οι υπάρχουσες παραλλαγές ομοσπονδιακής Προσαρμογής Χαμηλής Τάξης παραμένουν χρονικά αναποτελεσματικές, καθώς βασίζονται σε στατικές συχνότητες συγχρονισμού που αγνοούν τη δυναμική της τοπικής εκπαίδευσης.

Για την ταυτόχρονη αντιμετώπιση και των δύο διαστάσεων, η παρούσα διπλωματική εργασία διερευνά τον συνδυασμό της Ομοσπονδιακής Δυναμικής Στάθμισης (FDA) με την ομοσπονδιακή προσαρμογή χαμηλής τάξης. Συγκεκριμένα, αναπτύσσουμε και διατυπώνουμε το πλαίσιο FDA-RoLoRA, έναν αλγοριθμικό σχεδιασμό που συνδυάζει τη δυναμική στάθμιση βάσει διασποράς με τη Στιβαρή Προσαρμογή Χαμηλής Τάξης (RoLoRA). Η μέθοδος αυτή εξαλείφει τη διγραμμική ανακολουθία συσσωμάτωσης (bilinear aggregation inconsistency) μέσω εναλλασσόμενων ενημερώσεων μεταξύ των πινάκων υποπροβολής και υπερπροβολής (down-projection and up-projection matrices), ενώ ρυθμίζει δυναμικά τη συχνότητα συγχρονισμού μέσω αποσυζευγμένων και δυναμικά ενημερούμενων κατωφλίων διασποράς ανά πίνακα. Αξιολογούμε το προτεινόμενο πλαίσιο στο σύνολο δοκιμών GLUE χρησιμοποιώντας το μοντέλο `roberta-base`.

# Acknowledgements

First and foremost, I would like to express my sincere gratitude to my supervisor, Assoc. Prof. Vasilis Samoladas, for his continuous guidance, insightful advice, and constructive feedback throughout this thesis. His courses in Distributed Systems and Machine Learning Systems were pivotal during my studies, providing the core foundation for this research.

I extend my sincere appreciation to the examination committee members, Prof. Antonios Deligiannakis and Prof. Minos Garofalakis, for their time, thorough review, and valuable observations on this work.

I am equally grateful to my friends for their support, encouragement, and the unforgettable moments shared throughout our university years in Chania.

Finally, I dedicate this thesis to my family. Their unconditional love, patience, and continuous support have been my strongest anchor throughout every step of this journey.

# Contents

**Abstract** i

**Περίληψη** iii

**Acknowledgements** v

**List of Figures** ix

**List of Tables** xi

**List of Abbreviations** xiii

**1 Introduction** 1
- 1.1 Federated Learning and Optimization . . . 1
- 1.2 Foundation Models and Parameter-Efficient Fine-Tuning . . . 2
- 1.3 Aggregation Inconsistency in Federated LoRA . . . 3
- 1.4 Robust Low-Rank Adaptation . . . 3
- 1.5 Proposed Method: FDA-RoLoRA . . . 4
- 1.6 Thesis Outline . . . 5

**2 Background** 7
- 2.1 Neural Networks and Optimization . . . 7
  - 2.1.1 Neural Networks . . . 7
  - 2.1.2 Gradient Descent . . . 8
  - 2.1.3 Stochastic Gradient Descent . . . 8
- 2.2 Foundation Models and the Transformer Architecture . . . 8
- 2.3 Distributed Machine Learning . . . 9
- 2.4 Federated Learning and Communication Bottlenecks . . . 10
  - 2.4.1 Federated Averaging . . . 11
  - 2.4.2 FedOpt: Server-Side Adaptive Optimization . . . 12
  - 2.4.3 Client Drift and Statistical Heterogeneity . . . 12
  - 2.4.4 Dirichlet-Based Data Partitioning . . . 13
  - 2.4.5 Communication Cost Quantification . . . 13
- 2.5 Parameter-Efficient Fine-Tuning . . . 14
  - 2.5.1 The PEFT Landscape . . . 14
  - 2.5.2 Low-Rank Adaptation (LoRA) . . . 14
  - 2.5.3 Structural and Dynamic Asymmetry in LoRA Matrices . . . 15
- 2.6 Federated Low-Rank Adaptation . . . 16
- 2.7 Robust Low-Rank Adaptation . . . 17

2.8 Federated Dynamic Averaging . . . 18
2.8.1 Geometric Monitoring Foundations . . . 18

**3 Methodology** **21**
3.1 Combining Dynamic Averaging with Federated LoRA . . . 21
3.2 Foundational Precursor Protocols: RoLoRA and FDA . . . 22
3.3 Notation and Hyperparameters . . . 24
3.4 FDA-RoLoRA Initialization and Subspace Toggling . . . 26
3.5 Local Training and Direct Subspace Variance . . . 26
3.6 Server-Coordinated Synchronization Halt . . . 27
3.7 Global Aggregation and Independent Threshold Evolution . . . 27
3.7.1 Foundational Midpoint Scaling from FDA-Opt . . . 27
3.7.2 From Canonical Midpoint to Tunable Epoch Setpoints . . . 28
3.8 Server Optimization and Aggregation Dynamics . . . 28
3.9 Architectural Exploration and Evolutionary Design Decisions . . . 29
3.9.1 Dense Product Reconstruction and SVD Factorization . . . 29
3.9.2 Single Shared Threshold vs. Decoupled Dual-Track . . . 29
3.9.3 Phase Lag in Threshold Smoothing (EMA and SMA) . . . 30

**4 Implementation** **31**
4.1 Software Architecture . . . 31
4.2 Data Partitioning . . . 32
4.3 Simulation Controls and Early Stopping . . . 32
4.4 Model Configuration . . . 33
4.5 Training Pipeline . . . 34
4.6 Deployment Environment . . . 35

**5 Evaluation** **37**
5.1 Experimental Setup and Benchmark Protocol . . . 37
5.2 Benchmark Suite 1: Homogeneous Environments . . . 38
5.3 Benchmark Suite 2: Statistical Heterogeneity . . . 50

**6 Conclusion and Future Work** **55**
6.1 Synthesis of Results and Methodological Takeaways . . . 55
6.2 Limitations . . . 55
6.3 Future Work . . . 56

**Bibliography** **59**

# List of Figures


2.1 Parameter Server Architecture for Asynchronous Distributed Learning. Workers independently pull the latest parameters and push computed gradients. 10

2.2 Chronological sequence of a Federated Learning communication round (broadcast, local client training, update collection, and server aggregation). Figure reproduced from Theologitis et al. [28]. . . . . 11

2.3 Low-Rank Adaptation (LoRA) schematic: decomposition of weight updates into low-rank matrices $A$ and $B$ alongside frozen pretrained weights $W$. . . 15

2.4 Robust Low-Rank Adaptation (RoLoRA) alternating optimization scheme. 17

5.1 Multi-panel optimization telemetry on MNLI ($C = 20, R_{\max} = 50, \eta_c = 0.1, \alpha = 1000.0$) comparing FDA-RoLoRA against static RoLoRA baselines under homogeneous consensus. . . . . 41

5.2 Multi-panel optimization telemetry on MNLI ($C = 50, R_{\max} = 30, \eta_c = 0.1, \alpha = 1000.0$) under expanded cohort scale $C = 50$. . . . . 42

5.3 Multi-panel optimization telemetry on QNLI ($C = 20, R_{\max} = 50, \eta_c = 0.1, \alpha = 1000.0$) comparing FDA-RoLoRA against static RoLoRA baselines under homogeneous consensus. . . . . 43

5.4 Multi-panel optimization telemetry on QNLI ($C = 50, R_{\max} = 30, \eta_c = 0.1, \alpha = 1000.0$) under expanded cohort scale $C = 50$. . . . . 44

5.5 Multi-panel optimization telemetry on QQP ($C = 20, R_{\max} = 50, \eta_c = 0.1, \alpha = 1000.0$) comparing FDA-RoLoRA against static RoLoRA baselines under homogeneous consensus. . . . . 46

5.6 Multi-panel optimization telemetry on QQP ($C = 50, R_{\max} = 30, \eta_c = 0.1, \alpha = 1000.0$) under expanded cohort scale $C = 50$. . . . . 47

5.7 Multi-panel optimization telemetry on SST-2 ($C = 20, R_{\max} = 50, \eta_c = 0.1, \alpha = 1000.0$) comparing FDA-RoLoRA against static RoLoRA baselines under homogeneous consensus. . . . . 48

5.8 Multi-panel optimization telemetry on SST-2 ($C = 50, R_{\max} = 30, \eta_c = 0.1, \alpha = 1000.0$) under expanded cohort scale $C = 50$. . . . . 49

5.9 Multi-panel optimization telemetry on MNLI under Dirichlet label skew ($\alpha = 0.5, C = 20, R_{\max} = 50, \eta_c = 0.1$) comparing FDA-RoLoRA against static baselines. . . . . 51

5.10 Multi-panel optimization telemetry on QNLI under Dirichlet label skew ($\alpha = 0.5, C = 20, R_{\max} = 50, \eta_c = 0.1$) comparing FDA-RoLoRA against static baselines. . . . . 53

5.11 Empirical local epoch allocation dynamics under severe statistical heterogeneity (Dirichlet label skew $\alpha = 0.5$), illustrating the closed-loop setpoint tracking dynamics of FDA-RoLoRA on MNLI and QNLI under Dirichlet label skew ($\alpha = 0.5$). . . . . 54
5.12 Convergence and client drift variance trajectories on MNLI and QNLI under severe Dirichlet statistical heterogeneity ($\alpha = 0.5$). Top panels present validation accuracy trajectories; bottom panels present client coordinate drift variance ($\sigma_t^2$). . . . . 54

# List of Tables

3.1 Summary of mathematical notation and system hyperparameters. . . . . 24

5.1 Detailed Telemetry Across All IID Configurations ($\alpha = 1000$): RoLoRA vs. FDA-RoLoRA across GLUE benchmarks. . . . . 39

5.2 Detailed Telemetry Across All Non-IID Configurations ($\alpha = 0.5$): RoLoRA vs. FDA-RoLoRA across GLUE benchmarks under Dirichlet label skew. . . 50

# List of Abbreviations

| Abbreviation | Description |
|---|---|
| **Adam** | Adaptive Moment Estimation |
| **AdamW** | Adam with Decoupled Weight Decay |
| **AMS** | Alon-Matias-Szegedy |
| **FDA** | Federated Dynamic Averaging |
| **FDA-Opt** | Federated Dynamic Averaging Optimizer |
| **FDA-RoLoRA** | Federated Dynamic Averaging with Robust Low-Rank Adaptation |
| **FedAvg** | Federated Averaging |
| **FedLoRA** | Federated Low-Rank Adaptation |
| **FedOpt** | Federated Optimization |
| **FFA-LoRA** | Federated Freeze-A Low-Rank Adaptation |
| **FL** | Federated Learning |
| **FLoRA** | Heterogeneous Federated Low-Rank Adaptation |
| **GLUE** | General Language Understanding Evaluation |
| **IID** | Independent and Identically Distributed |
| **LLM** | Large Language Model |
| **LoRA** | Low-Rank Adaptation |
| **MNLI** | Multi-Genre Natural Language Inference |
| **non-IID** | Non-Independent and Identically Distributed |
| **PEFT** | Parameter-Efficient Fine-Tuning |
| **RoLoRA** | Robust Low-Rank Adaptation |
| **SGD** | Stochastic Gradient Descent |
| **SLoRA** | Sparse Low-Rank Adaptation |
| **SST-2** | Stanford Sentiment Treebank (Binary) |

# Chapter 1

# Introduction

Federated Learning (FL) [12,21] has established itself as the leading paradigm for training machine learning models across decentralized data owners without requiring the centralization of raw user data. By enabling participating nodes (ranging from resource-constrained edge devices to institutional data silos such as hospitals and financial institutions) to compute updates locally and share only parameter tensors with a coordinating server, FL enables collaborative optimization while respecting data privacy regulations (e.g., GDPR, HIPAA) and proprietary commercial boundaries.

However, the iterative nature of federated optimization introduces a severe communication bottleneck. Historically, research in distributed machine learning has tackled this challenge primarily through temporal and algorithmic mechanisms designed for standard neural architectures. Yet, the recent paradigm shift toward massive foundation models has created an entirely new spatial bottleneck, rendering classical federated optimization techniques insufficient on their own.

## 1.1 Federated Learning and Optimization

In canonical federated optimization, the central communication bottleneck is dictated by the synchronization frequency between clients and the central server. The foundational Federated Averaging (FedAvg) algorithm [21] coordinates training by having clients execute a fixed, predetermined number of local stochastic gradient descent (SGD) epochs ($E$) before transmitting their updated parameter vectors to the server for aggregation.

While conceptually simple, rigid synchronization cadences suffer from fundamental inefficiencies under real-world conditions:

- **Statistical Heterogeneity and Client Drift:** In decentralized environments, local client datasets are inherently non-identically and independently distributed (non-IID) [13,16]. When clients execute multiple uncoordinated local epochs, their local models drift toward local empirical minima that diverge sharply from the global optimization trajectory.

- **Temporal Redundancy:** Enforcing a static schedule remains blind to actual client convergence. In phases where local updates are minor or homogeneous, transmitting parameters after every fixed interval wastes critical network bandwidth on redundant updates. Conversely, when clients diverge rapidly, static schedules delay aggregation, exacerbating model divergence.

To improve federated optimization, adaptive server-side optimizers were introduced under the Federated Optimization (FedOpt) framework [23]. By interpreting aggregated client updates as pseudo-gradients and applying momentum-based tracking (such as FedAdam and FedYogi), FedOpt stabilizes the global optimization trajectory against client drift.

Subsequently, to resolve the rigid cadence of static rounds, Federated Dynamic Averaging (FDA-Opt) [8, 28] introduced variance-driven dynamic synchronization. Rather than synchronizing after fixed epochs, FDA-Opt continuously monitors the pairwise drift and variance across participating clients. Synchronization is triggered dynamically only when parameter variance breaches an adaptive drift threshold ($\theta$). By synchronizing strictly when mathematically necessary, FDA-Opt proved that dynamic temporal throttling can dramatically reduce total communication rounds without sacrificing final accuracy.

Crucially, however, FedAvg, FedOpt, and FDA-Opt all share a common architectural assumption: *the transmission and synchronization of full-rank model parameter vectors.*

## 1.2 Foundation Models and Parameter-Efficient Fine-Tuning

In recent years, the machine learning landscape has undergone an unprecedented transformation driven by large-scale pre-trained foundation models and Large Language Models (LLMs) scaling to billions of parameters [4, 5]. Adapting these foundation models to specialized domain tasks (e.g., clinical diagnosis, legal discovery, financial fraud detection) is of paramount importance.

Yet, this shift in model scale fundamentally breaks the physical viability of full-parameter federated learning:

- Synchronizing a modern 7-billion parameter language model (such as LLaMA-2-7B [29]) in 16-bit half-precision requires transmitting approximately **14 Gigabytes** of parameter data per client per round.
- In a cross-silo federated setting with 50 participating institutions over a modest horizon of 100 communication rounds, total network transfer exceeds **70 Terabytes**, an operational volume that is wholly intractable over standard wide-area network (WAN) connections.
- Even for moderately sized encoders such as RoBERTa-Large [19] (355M parameters), a single full-parameter round requires $\sim$1.4 GB per client, swamping edge up-links.

To overcome this spatial bottleneck, Parameter-Efficient Fine-Tuning (PEFT) has emerged as an indispensable methodology. Among PEFT strategies, Low-Rank Adaptation (LoRA) [11] has become the de facto standard. LoRA keeps the large pre-trained weight matrix $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ frozen, and factorizes task-specific parameter updates into two low-rank matrices:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA \tag{1.1}$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ denotes the down-projection matrix, $B \in \mathbb{R}^{d_{\text{out}} \times r}$ denotes the up-projection matrix, and the rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$ (typically $r \in \{4, 8, 16\}$). Because only $A$ and $B$ are updated during training, LoRA compresses the volume of communicated parameters by over 99%, providing a powerful spatial solution for edge and silo environments.

## 1.3 Aggregation Inconsistency in Federated LoRA

Given the spatial efficacy of LoRA, directly integrating low-rank adapters into federated learning (often termed Federated LoRA (FedLoRA) or FedIT [33, 35]) appears to be a natural path forward. In naive FedLoRA, each client $i \in \{1, \ldots, N\}$ initializes identical low-rank matrices, computes local gradient updates on private data, and transmits only its trained adapter matrices $A_i$ and $B_i$ to the central server.

However, aggregating independently trained low-rank factors via standard FedAvg introduces a fundamental mathematical inconsistency: *LoRA aggregation inconsistency* (or aggregation discordance) [33]. Because matrix multiplication is non-linear and non-commutative, averaging the factor matrices independently across clients fails to reconstruct the true average of their composite weight updates:

$$\frac{1}{N}\sum_{i=1}^{N}(B_i A_i) \neq \left(\frac{1}{N}\sum_{i=1}^{N} B_i\right)\left(\frac{1}{N}\sum_{i=1}^{N} A_i\right) \tag{1.2}$$

When the central server independently averages $A_i$ and $B_i$, the composite product of the averaged matrices generates cross-term interference vectors that do not exist in any client's local optimization trajectory. This bilinear distortion collapses the low-rank representation, induces severe gradient cancellation, and destabilizes global convergence, frequently causing fine-tuning to perform worse than static heuristics or simple localized training.

## 1.4 Robust Low-Rank Adaptation

To resolve this bilinear aggregation discordance, several structural workarounds have been explored in recent literature:

- **Full Weight Reconstruction:** Clients could theoretically reconstruct the dense update $\Delta W_i = \frac{\alpha}{r} B_i A_i$ before transmission, or the server could perform Singular Value Decomposition (SVD) on the aggregated full matrix [33]. However, this completely nullifies the spatial communication savings of LoRA and imposes prohibitive computational overhead.

- **Static Down-Projection Freezing (FFA-LoRA):** FFA-LoRA [27] permanently freezes the down-projection matrix $A$ as a random projection across all clients, allowing clients to train and transmit only the up-projection matrix $B$. While this linearizes aggregation ($\frac{1}{N}\sum_{i=1}^{N} B_i A = \left(\frac{1}{N}\sum_{i=1}^{N} B_i\right) A$), permanently freezing half of the parameter degrees of freedom severely compromises adapter capacity and expressive power.

- **Heterogeneous Rank Assembly (FLoRA):** FLoRA [32] attempts to support variable client ranks by stacking updates, but still incurs heavy reconstruction overhead on the aggregation server.

An effective solution to this inconsistency without sacrificing model expressiveness is **Robust Low-Rank Adaptation (RoLoRA)** [6]. RoLoRA adopts an *alternating coordinate optimization* strategy. Instead of updating both factor matrices simultaneously, training alternates in structured multi-round blocks:

1. In Phase A, matrix $A$ is frozen globally across all clients, and only matrix $B$ is trained locally and aggregated.

2. In Phase B, the updated matrix $B$ is frozen globally across all clients, and only matrix $A$ is trained locally and aggregated.

Because one coordinate factor is held globally identical across all clients during any given aggregation phase, the averaging operation becomes strictly linear:

$$\frac{1}{N}\sum_{i=1}^{N}(B_i A_{\text{frozen}}) = \left(\frac{1}{N}\sum_{i=1}^{N} B_i\right) A_{\text{frozen}} \tag{1.3}$$

By toggling the active subspace over alternating cycles, RoLoRA preserves the full representational rank and expressive capacity of standard LoRA while restoring mathematically exact federated averaging.

## 1.5 Proposed Method: FDA-RoLoRA

With RoLoRA restoring mathematical linearity to federated low-rank aggregation, a natural research question arises: *Can we now unify the temporal adaptivity of Federated Dynamic Averaging with the spatial efficiency of alternating low-rank adaptation?*

While RoLoRA successfully solves the spatial aggregation inconsistency, existing RoLoRA formulations remain constrained by **rigid, predetermined synchronization cadences** (fixed local steps or epochs). Clients must synchronize their active matrix after arbitrary epoch boundaries, inheriting all the temporal redundancies and drift penalties of classical FedAvg.

Conversely, while FDA-Opt [28] pioneered variance-driven dynamic synchronization, it was formulated exclusively for monolithic full-rank neural networks. Simply applying FDA-Opt directly to RoLoRA is impossible due to three fundamental obstacles:

1. **Asymmetric Coordinate Geometries:** The down-projection matrix $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and up-projection matrix $B \in \mathbb{R}^{d_{\text{out}} \times r}$ fulfill entirely different functional roles (compression vs. expansion). Furthermore, standard initialization schemes ($A \sim \mathcal{N}(0, \sigma^2)$ and $B = 0$) create highly divergent gradient dynamics, spectral velocities, and parameter drift magnitudes. A single scalar threshold cannot govern both spaces.

2. **Subspace Coordinate Variance:** If client variance were computed by projecting updates back to full-rank matrices ($\Delta W = \frac{\alpha}{r} BA$), all computational and memory benefits of PEFT would be lost. Client drift must be measured *directly within the low-rank coordinate subspace.*

3. **Geometric Cross-Contamination:** In alternating optimization, threshold evolution must not leak variance metrics between coordinate tracks. Dynamic thresholds for matrix $A$ must evolve independently from matrix $B$.

To bridge this divide, this thesis introduces **FDA-RoLoRA**, a unified federated framework that combines alternating coordinate optimization with direct subspace dynamic averaging via a **Decoupled Dual-Track Threshold Mechanism**. By maintaining independent variance monitoring and threshold evolution tracks ($\theta_A$ and $\theta_B$), FDA-RoLoRA achieves simultaneous spatial compression, exact mathematical aggregation linearity, and

dynamic communication throttling without projecting updates back to the full-rank parameter space.

To evaluate this approach, we implement an end-to-end multi-process simulation testbed in PyTorch and Hugging Face PEFT, supporting Dirichlet-based non-IID data partitioning across transformer foundation models. Through this framework, we examine the convergence characteristics, communication savings, and parameter sensitivity of dynamic subspace averaging.

## 1.6 Thesis Outline

The remainder of this diploma thesis is organized as follows:

- **Chapter 2** establishes the theoretical foundations underpinning this work, reviewing stochastic gradient descent, foundation models, the Transformer architecture, client drift under statistical heterogeneity, server-side adaptive optimizers (FedOpt), Low-Rank Adaptation (LoRA), Robust LoRA (RoLoRA), and Federated Dynamic Averaging (FDA).
- **Chapter 3** details the mathematical formulation of the proposed FDA-RoLoRA methodology, describing the alternating coordinate toggling protocol, direct subspace variance computation, independent halting criteria, dual-track threshold evolution strategies, and server-side aggregation.
- **Chapter 4** describes the software architecture and simulation platform, detailing PyTorch adapter injection, Dirichlet non-IID data partitioning, in-silico sequential GPU client simulation, and cluster execution workflows on dedicated accelerator infrastructure.
- **Chapter 5** presents the empirical evaluation framework and experimental results across 173 runs on the GLUE benchmark using `roberta-base`, analyzing convergence behaviors, anti-collapse safety braking under severe label skew, and communication payload reductions.
- **Chapter 6** summarizes the core contributions and methodological takeaways of the thesis, discusses practical engineering limitations, and outlines promising directions for future research.

# Chapter 2

# Background

This chapter provides the foundational concepts necessary to understand the proposed methodology. We begin by reviewing the fundamental principles of neural networks and optimization via gradient descent. We then discuss the architectural scaling of foundation models, the transfer learning paradigm, and the resulting communication bottlenecks inherent in decentralized distributed systems. Next, we examine the core federated learning framework (including client drift under statistical heterogeneity, the Dirichlet partitioning methodology, and server-side adaptive optimization) before quantifying the communication costs that motivate parameter-efficient methods. We then survey the parameter-efficient fine-tuning landscape, focusing on Low-Rank Adaptation and the asymmetric properties of its factor matrices, and analyze the spatial aggregation inconsistency problem that arises when integrating low-rank adapters into federated training. Finally, we explore the geometric monitoring foundations and temporal optimization mechanisms underlying Federated Dynamic Averaging, establishing the theoretical mechanics that underpin FDA-RoLoRA presented in Chapter 3.

## 2.1 Neural Networks and Optimization

### 2.1.1 Neural Networks

Supervised learning with neural networks is a cornerstone of modern machine learning, enabling systems to learn complex mappings from input data to desired outputs. In this setting, the objective is to learn a function parameterized by a neural network that generalizes well to unseen data.

In the forward pass, an input vector $x$ is processed through the network layers to produce a prediction. This can be formally denoted as:

$$\hat{y} = f_w(x) \tag{2.1}$$

where $f_w$ represents the neural network parameterized by a set of weights $w$. The goal of the learning process is to adjust $w$ such that the predictions $\hat{y}$ closely match the true labels $y$.

To quantify the performance of the network, we define a loss function. For a dataset containing $n$ samples, the empirical risk or average loss $\mathcal{L}(w)$ is computed as:

$$\mathcal{L}(w) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_w(x_i), y_i) \tag{2.2}$$

where $\ell(\cdot, \cdot)$ is a sample-wise loss function (e.g., cross-entropy for classification). Minimizing this loss function with respect to the parameters $w$ is the central optimization problem in training neural networks.

### 2.1.2 Gradient Descent

The minimization of the empirical risk is typically performed using gradient-based optimization. The gradients of the loss function with respect to the network weights, denoted as $\nabla_w \mathcal{L}(w)$, are computed efficiently using the backpropagation algorithm. Backpropagation systematically applies the chain rule of calculus from the output layer back to the input layer, distributing the error gradients across all intermediate parameters.

With the gradients computed, the network weights are iteratively updated using Gradient Descent (GD). The fundamental GD update rule at training step $t$ is defined as:

$$w_{t+1} = w_t - \eta \nabla_w \mathcal{L}(w_t) = w_t - \eta \frac{1}{n} \sum_{i=1}^{n} \nabla_w \ell(f_{w_t}(x_i), y_i) \tag{2.3}$$

where $\eta$ is the learning rate, a hyperparameter controlling the step size of the update. In standard GD, the true gradient is computed over the entire dataset of size $n$ at each iteration.

### 2.1.3 Stochastic Gradient Descent

In practice, computing the full gradient over the entire dataset is computationally prohibitive for large datasets. Therefore, Stochastic Gradient Descent (SGD) and its mini-batch variant are universally employed. In mini-batch SGD, the true gradient is approximated using a small subset or mini-batch of $m$ samples (where $m \ll n$). The update rule becomes:

$$w_{t+1} = w_t - \eta \frac{1}{m} \sum_{j=1}^{m} \nabla_w \ell(f_{w_t}(x_{i_j}), y_{i_j}) \tag{2.4}$$

This introduces stochasticity into the optimization process, which not only accelerates each training iteration by reducing the computational cost per step, but can also aid the model in escaping shallow local minima.

While vanilla SGD is effective, modern deep learning often utilizes adaptive optimization algorithms, such as Adam. These optimizers maintain moving averages of past gradients and squared gradients to scale the learning rate on a per-parameter basis, accelerating convergence and improving stability. As we will discuss in later sections, the concepts behind adaptive optimizers are highly relevant when addressing the challenges of server-side optimization in distributed settings.

## 2.2 Foundation Models and the Transformer Architecture

In recent years, the machine learning paradigm has undergone a profound shift, transitioning from task-specific neural networks to large-scale pre-trained *foundation models* [4]. Foundation models, which include Large Language Models (LLMs) such as GPT-3 [5], RoBERTa [19], and Llama [29], are trained on vast unlabeled datasets using self-supervised

learning, enabling them to be adapted to a broad range of downstream tasks with minimal structural modifications.

At the core of virtually all modern foundation models is the Transformer architecture introduced by Vaswani et al. [30]. Unlike traditional Recurrent Neural Networks (RNNs) or Convolutional Neural Networks (CNNs) that process sequential data step-by-step or through localized receptive fields, the Transformer relies fundamentally on the multi-head self-attention mechanism. For an input representation matrix $X \in \mathbb{R}^{n \times d}$, self-attention projects $X$ into Query ($Q$), Key ($K$), and Value ($V$) representations using parameter matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{2.5}$$

This formulation allows the network to capture pairwise interactions between all tokens in a sequence simultaneously, enabling high computational parallelization during training.

However, the empirical success of foundation models has been driven by massive parameter scaling. Modern architectures frequently range from hundreds of millions (e.g., RoBERTa-Large with 355M parameters) to tens or hundreds of billions of trainable parameters (e.g., Llama-2 with 7B to 70B parameters) [29]. This unprecedented scale introduces severe computational and memory challenges. Full fine-tuning (updating all weights $w \in \mathbb{R}^d$ during downstream optimization) requires storing activation states, gradients, and optimizer momentum buffers for billions of parameters simultaneously. Consequently, deploying and adapting foundation models shifts machine learning from single-device training to distributed infrastructures and parameter-efficient adaptation paradigms.

The practical utility of foundation models is realized through the *transfer learning* paradigm. During the first phase, a model is pre-trained on a massive, general-purpose corpus using self-supervised objectives (e.g., masked language modeling for encoder models or next-token prediction for autoregressive models), learning broad linguistic and semantic representations. In the second phase, this pre-trained model is *fine-tuned* on a smaller, task-specific labeled dataset (such as sentiment classification, natural language inference, or instruction following [22]) by continuing gradient-based optimization on the downstream objective. The key insight is that the representations learned during pre-training transfer effectively to downstream tasks, enabling strong performance with far less task-specific data than training from scratch. However, standard fine-tuning still requires updating all $d$ parameters, which motivates the parameter-efficient approaches discussed in subsequent sections.

## 2.3 Distributed Machine Learning

As the complexity of tasks and the volume of available data have grown, both datasets and neural network architectures have become too large to be efficiently processed on a single machine. This computational bottleneck has motivated the development of distributed machine learning systems, where the training process is partitioned across multiple worker nodes or machines. A comprehensive landscape of modern parallel and distributed learning methodologies, including detailed taxonomies of synchronization models and parallelism paradigms, is surveyed by Subasi et al. [26].

Distributed training strategies generally fall into two categories: data parallelism and model parallelism [26]. In data parallelism, the dataset is divided into partitions dis-

tributed across the workers, while each worker maintains a complete replica of the model parameters. Conversely, in model parallelism, a single model is too large to fit in the memory of a single machine, so its layers or components are distributed across multiple workers, while they all process the same data stream. Data parallelism is the most prevalent approach in scenarios scaling across large numbers of nodes.

In a standard data-parallel setup using synchronous SGD, all worker nodes independently compute gradients on their local data batches. Once all workers finish computing their local gradients, they synchronize by aggregating them, typically using an all-reduce operation. The globally averaged gradient is then used to update the model parameters across all nodes simultaneously. While mathematically equivalent to centralized mini-batch SGD with a larger batch size, synchronous SGD suffers from the straggler effect: the entire system must wait for the slowest worker to complete its computation before proceeding to the next step.

To mitigate the straggler problem, asynchronous SGD was introduced, often implemented via a parameter server architecture (illustrated in Figure 2.1). In this paradigm, workers independently fetch the latest model parameters from a central server, compute gradients on their local data, and push the gradients back to the server without waiting for other workers. The server updates the model immediately upon receiving a gradient. However, this lack of synchronization introduces the staleness problem: a worker might compute gradients based on an older version of the model, and by the time it pushes its update, the server's model has already been updated by other workers. These stale gradients can introduce noise, destabilize training, and degrade final model performance.

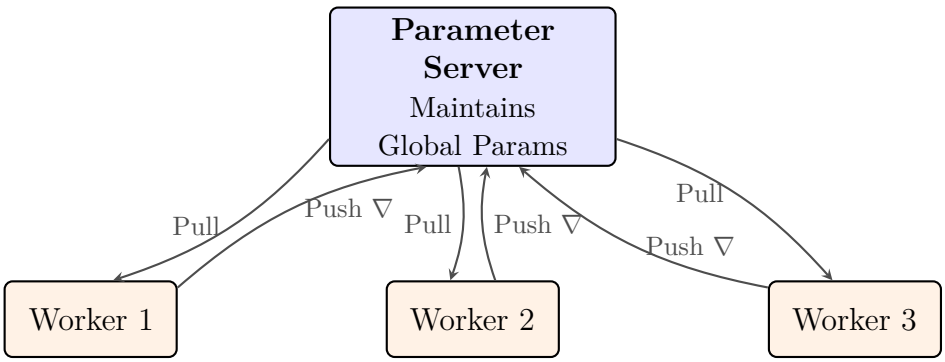



Figure 2.1: Parameter Server Architecture for Asynchronous Distributed Learning. Workers independently pull the latest parameters and push computed gradients.

The evolution of these distributed optimization techniques laid the groundwork for scaling machine learning. However, traditional distributed systems assume reliable communication networks, homogeneous compute nodes, and centrally accessible data. The emergence of privacy concerns and the proliferation of edge devices necessitated a paradigm shift, giving rise to Federated Learning as a privacy-preserving variant of distributed learning that addresses the unique challenges of decentralized data.

## 2.4 Federated Learning and Communication Bottlenecks

Federated Learning significantly diverges from classical distributed learning by focusing on privacy preservation across decentralized edge devices. Unlike traditional distributed

setups with centrally managed data and homogeneous reliable workers, FL must navigate highly heterogeneous local datasets, unreliable network connections, and computationally constrained clients.

### 2.4.1 Federated Averaging

Federated Learning [12] enables collaborative machine learning across a set of decentralized clients $K$ without centralizing private data. Figure 2.2 illustrates the typical FL lifecycle, comprising local training and global aggregation. Each client $k \in K$ retains its private dataset $\mathcal{D}_k$. The collective objective is to solve the distributed optimization problem:

$$\min_{w \in \mathbb{R}^d} F(w) = \frac{1}{|K|} \sum_{k \in K} F_k(w) \tag{2.6}$$

where $F_k(w) = \mathbb{E}_{x \sim \mathcal{D}_k}[\ell(w; x)]$ represents the local objective function of client $k$ over a loss function $\ell(w; x)$ parameterized by the model weights $w \in \mathbb{R}^d$.

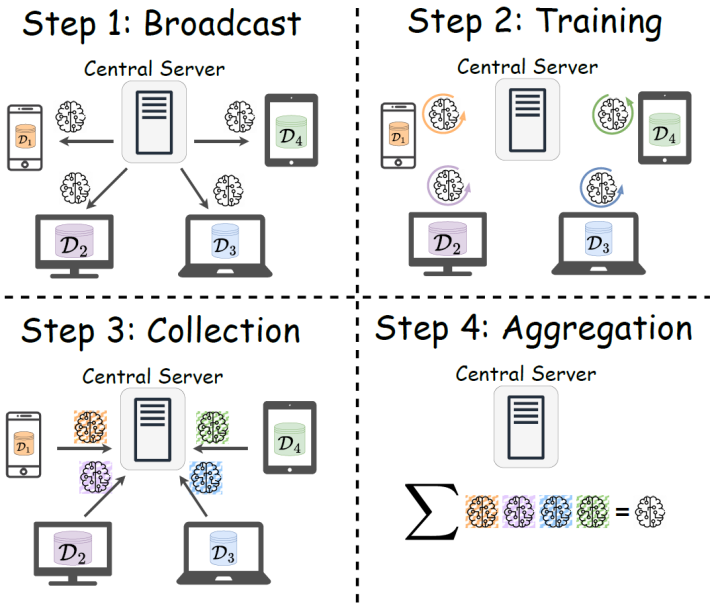



Figure 2.2: Chronological sequence of a Federated Learning communication round (broadcast, local client training, update collection, and server aggregation). Figure reproduced from Theologitis et al. [28].

To solve this problem, the standard Federated Averaging (FedAvg) algorithm [21] operates in communication rounds. At round $t$, the server selects a client cohort $S_t \subset K$ and broadcasts the global model $w_t$. The selected clients initialize their local models as $w_{t,0}^{(k)} = w_t$ and perform $\tau$ local steps of Stochastic Gradient Descent (SGD):

$$w_{t,j+1}^{(k)} = w_{t,j}^{(k)} - \eta \nabla \ell(w_{t,j}^{(k)}; x) \tag{2.7}$$

After training, clients transmit their local updates $\Delta_t^{(k)} = w_{t,\tau}^{(k)} - w_t$ to the server, which aggregates them to compute the next global model:

$$w_{t+1} = w_t + \frac{1}{|S_t|} \sum_{k \in S_t} \Delta_t^{(k)} \tag{2.8}$$

### 2.4.2 FedOpt: Server-Side Adaptive Optimization

For modern large foundation models, transmitting full weight vectors $w \in \mathbb{R}^d$ introduces severe spatial communication overhead. Furthermore, client-level statistics are often noisy, and clients are typically stateless, preventing the direct use of adaptive optimizers on edge devices. To address this, the FedOpt framework [23] shifts adaptive optimization to the server. Leveraging the adaptive optimization principles (e.g., Adam) discussed in Section 2.1, FedOpt allows the server to aggregate client parameter displacements $\Delta_t^{(k)} = w_{t,\tau}^{(k)} - w_t$ across the cohort $S_t$:

$$\Delta_t = \frac{1}{|S_t|} \sum_{k \in S_t} \Delta_t^{(k)} \tag{2.9}$$

Because local updates move along descent trajectories, $\Delta_t$ represents the average parameter displacement. In gradient-based server optimizers (e.g., FedAdam, FedAdamW), this quantity defines an effective server pseudo-gradient with an inverted sign:

$$\nabla_{\text{pseudo},t} = -\Delta_t = -\frac{1}{|S_t|} \sum_{k \in S_t} \Delta_t^{(k)} \tag{2.10}$$

ensuring that standard gradient-descent update steps ($w_{t+1} = w_t - \eta_s \nabla_{\text{pseudo},t}$, as formalized in Chapter 3) step in the direction of cumulative local progress, or equivalently through an additive displacement step:

$$w_{t+1} = \text{ServerOpt}(w_t, \Delta_t) \tag{2.11}$$

The motivation for server-side adaptive optimization in federated learning is distinct from its role in centralized training. In centralized settings, adaptive optimizers like Adam [14] accelerate convergence by maintaining per-parameter learning rate scaling. In federated settings, the pseudo-gradient $\nabla_{\text{pseudo},t}$ suffers from compounded noise: it is computed from a sampled client subset $S_t \subset K$ rather than all clients, and each client's local update reflects multiple SGD steps on heterogeneous data rather than a single gradient evaluation. Server-side momentum buffers smooth these noisy pseudo-gradients across rounds, providing a more stable optimization trajectory.

Furthermore, federated clients are inherently *stateless*: they do not persist optimizer state (momentum, variance estimates) between communication rounds, as each round begins with a fresh initialization from the global model. This precludes the use of adaptive optimizers at the client level across rounds, making the server the only entity capable of maintaining long-term optimization history. Additionally, in parameter-efficient adaptation regimes (detailed in Section 2.5), decoupled weight decay (AdamW [20]) at the server provides implicit regularization on adapter parameters, helping prevent overfitting to the heterogeneous local distributions encountered during federated training.

While FedOpt stabilizes training and accelerates convergence, it still relies on rigid, static synchronization schedules (fixed $\tau$), ignoring local optimization dynamics and leading to temporal communication inefficiencies.

### 2.4.3 Client Drift and Statistical Heterogeneity

A defining challenge in federated learning is the statistical heterogeneity of client data distributions. In practice, client datasets $\mathcal{D}_k$ are rarely independent and identically distributed (IID). Instead, they exhibit various forms of heterogeneity: *label skew*, where

different clients possess data from different subsets of classes; *quantity skew*, where the number of local training samples varies significantly across clients; and *feature skew*, where the underlying input distributions differ even for the same label [37].

Under non-IID data, the local objective functions $F_k(w)$ may have significantly different minimizers. When clients perform multiple steps of local SGD on their heterogeneous data, their parameters drift toward distinct local optima that diverge from the global optimum. This phenomenon, known as *client drift* [13], causes the aggregated global update to deviate from the true gradient direction, degrading convergence speed and final model quality.

The severity of client drift is formally analyzed by Li et al. [17], who establish convergence bounds for FedAvg under non-IID data that depend on the degree of gradient dissimilarity across clients. Various drift-mitigation strategies have been proposed: FedProx [16] adds a proximal regularization term to constrain local updates to remain close to the global model, while SCAFFOLD [13] introduces variance reduction through control variates that correct for client drift at each local step. These methods highlight that *monitoring and controlling the divergence between local and global model states is fundamental to stable federated optimization*, a principle that directly motivates the variance-driven synchronization approach explored later in this chapter.

### 2.4.4 Dirichlet-Based Data Partitioning

To rigorously evaluate federated algorithms under controlled non-IID conditions, the research community has adopted the Dirichlet distribution as the standard tool for synthetic data partitioning [10, 34, 37]. For a classification task with $C$ classes and $K$ clients, each client $k$ receives a class proportion vector $p_k \sim \mathrm{Dir}(\alpha)$, where $\mathrm{Dir}(\alpha)$ denotes the symmetric Dirichlet distribution with concentration parameter $\alpha > 0$. Training samples of each class are then allocated to clients according to these sampled proportions.

The concentration parameter $\alpha$ provides a single scalar knob to control the degree of heterogeneity. As $\alpha \to 0$, the sampled proportion vectors become increasingly peaked, producing extreme label skew where individual clients may hold data from only a single class. Conversely, as $\alpha \to \infty$, the proportions converge to a uniform distribution, recovering the IID setting. Intermediate values (e.g., $\alpha = 0.5$ or $\alpha = 1.0$) produce moderate heterogeneity that reflects realistic scenarios where clients have partial but imbalanced class coverage. This configurability makes the Dirichlet partitioning scheme essential for conducting systematic ablation studies across varying degrees of statistical adversity.

### 2.4.5 Communication Cost Quantification

To make the communication bottleneck concrete, consider the per-round transmission cost for foundation models. For a transformer backbone such as `roberta-base` (125 M parameters), transmitting the full model weights in standard 32-bit floating point precision requires approximately 500 MB per client per synchronization round (or $\sim$ 250 MB in 16-bit half-precision). Across a modest cohort of 10 clients over a 100-round federated training run, this results in up to 500 GB of raw model transfers over uplink-constrained edge networks. In multi-billion parameter foundation architectures (such as Llama-2-7B with $\approx 7 \times 10^9$ parameters), a single full-model synchronization demands approximately 28 GB in FP32 per client per round, which is completely intractable in distributed federated settings.

To overcome this prohibitive uplink requirement, parameter-efficient fine-tuning (PEFT) methods restrict trainable parameters to compact modular adapters. In particular, Low-Rank Adaptation (LoRA, formalised in Section 2.5) decomposes weight updates into low-rank factor matrices. When targeting the attention projection matrices of `roberta-base` with a compact adapter rank $r = 4$ (typically $r \in \{4, 8\}$), the communicated payload drops from 500 MB to under 1 MB per client per round, achieving a spatial compression ratio exceeding 500×. This substantial reduction demonstrates why low-rank parametrization is essential to make federated training of foundation models feasible across bandwidth-constrained edge networks.

## 2.5 Parameter-Efficient Fine-Tuning

The massive scale of foundation models renders traditional full fine-tuning (updating all $d$ parameters for a downstream task) increasingly impractical. Full fine-tuning requires storing not only the model weights but also the corresponding activation states, gradient tensors, and optimizer momentum buffers (e.g., first and second moment estimates for Adam) for every trainable parameter. For a model with $d$ parameters in mixed-precision training, the aggregate memory footprint can exceed $16d$ bytes, placing models with hundreds of millions or billions of parameters well beyond the capacity of commodity client hardware. Parameter-Efficient Fine-Tuning (PEFT) addresses this challenge by updating only a small subset of parameters while keeping the vast majority of pre-trained weights frozen.

### 2.5.1 The PEFT Landscape

Several families of PEFT techniques have been developed to balance parameter efficiency with representational capacity:

- **Adapter Layers**: Introduced by Houlsby et al. [9], adapter modules inject small, learnable bottleneck layers between existing transformer feed-forward and self-attention projections. While parameter-efficient, they introduce structural changes that incur non-negligible inference latency.
- **Prefix Tuning**: Proposed by Li and Liang [18], prefix tuning prepends learnable, continuous task-specific vectors directly to the key and value sequences of transformer attention layers, steering model behavior without modifying original parameters. However, prepending prefixes reduces the effective sequence length available for downstream context.
- **Prompt Tuning**: A further simplification of prefix tuning that prepends virtual token embeddings strictly to the input layer, offering high parameter economy but often exhibiting slower optimization convergence on complex tasks.

### 2.5.2 Low-Rank Adaptation (LoRA)

Among parameter-efficient adaptation paradigms, Low-Rank Adaptation (LoRA) [11] has emerged as the prevailing standard for both centralized and distributed environments. LoRA operates on the foundational hypothesis that weight updates during downstream task adaptation occupy a low intrinsic rank [1].

Rather than modifying a dense pre-trained weight matrix $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ directly, LoRA parameterizes the weight update $\Delta W$ through a low-rank matrix decomposition:

$$\Delta W = \frac{\alpha}{r} BA \tag{2.12}$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ denotes the down-projection matrix, $B \in \mathbb{R}^{d_{\text{out}} \times r}$ denotes the up-projection matrix, rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$, and $\alpha$ is a constant scaling hyperparameter. The forward pass of a modified layer computes:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} BAx \tag{2.13}$$

Throughout fine-tuning, the pre-trained backbone weights $W_0$ remain completely frozen, receiving zero gradient updates. Only the compact factor matrices $A$ and $B$ are optimized. Under the canonical initialization scheme, $A$ is initialized with random Gaussian values $\mathcal{N}(0, \sigma^2)$, while $B$ is initialized to zero, ensuring that $\Delta W = \frac{\alpha}{r} BA = 0$ at training onset and the model's pre-trained predictions are exactly preserved prior to adaptation. The scaling factor $\frac{\alpha}{r}$ stabilizes the optimization dynamics when varying the adapter rank $r$, typically set such that $\alpha = 2r$. Figure 2.3 illustrates this architecture.

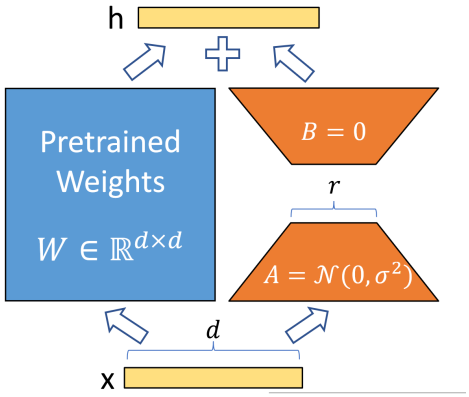



Figure 2.3: Low-Rank Adaptation (LoRA) schematic: decomposition of weight updates into low-rank matrices $A$ and $B$ alongside frozen pretrained weights $W$.

Unlike adapter layers, LoRA introduces zero additional inference latency: upon completion of training, the factorized update $\frac{\alpha}{r} BA$ can be folded directly into the pre-trained weights ($W_{\text{final}} = W_0 + \frac{\alpha}{r} BA$) via simple matrix addition. In centralized settings, extensions such as QLoRA [7] (which integrates 4-bit NormalFloat base quantization) and AdaLoRA [36] (which dynamically allocates rank budgets across layers) have further highlighted the versatility of the low-rank paradigm.

### 2.5.3 Structural and Dynamic Asymmetry in LoRA Matrices

An essential, frequently overlooked property of the LoRA decomposition is the inherent asymmetry between its two factor matrices. Functionally, the down-projection matrix $A \in \mathbb{R}^{r \times d_{\text{in}}}$ compresses input features from high-dimensional representations into the low-rank bottleneck, whereas the up-projection matrix $B \in \mathbb{R}^{d_{\text{out}} \times r}$ expands compressed representations back to the original dimensionality. These distinct projection roles produce asymmetric gradient dynamics during backpropagation.

Furthermore, the canonical initialization scheme creates an acute temporal asymmetry: because $B = 0$ at initialization, the initial gradients with respect to $A$ ($\nabla_A \mathcal{L} \propto B^\top \nabla_h \mathcal{L}$) are initially zero, whereas the gradients with respect to $B$ ($\nabla_B \mathcal{L} \propto \nabla_h \mathcal{L}(Ax)^\top$) immediately receive non-zero projections from the Gaussian-initialized $A$. Consequently, the optimization trajectories, convergence velocities, and parameter variance profiles of $A$ and $B$ diverge substantially during early training rounds, with matrix $B$ typically manifesting significantly higher drift. In federated learning, this asymmetry implies that treating both coordinate matrices identically during aggregation or synchronization scheduling is fundamentally suboptimal.

## 2.6 Federated Low-Rank Adaptation

Having established standard LoRA and the core FL framework, combining the two paradigms, formalized as Federated Low-Rank Adaptation (FedLoRA) [33], is a natural evolution to reduce the massive communication costs of federated learning. In a typical FL setting, transmitting full model updates $w \in \mathbb{R}^d$ imposes severe network strain. By adopting LoRA, clients only train and transmit the low-rank matrices $A$ and $B$, which drastically reduces the spatial communication overhead of federated synchronization.

A comprehensive survey of FedLoRA [33] categorizes the primary challenges in this domain into distributed learning stability, data heterogeneity, and efficiency. To address these challenges, several frameworks have been proposed. For instance, FedIT [35] applies standard federated averaging directly to LoRA parameters. To mitigate communication costs further and ensure exact updates, FFA-LoRA [27] freezes the down-projection matrices $A$ across all clients, allowing clients to update and transmit only the up-projection matrices $B$. Other approaches, such as FLoRA [32], support heterogeneous-rank adapters among clients to accommodate varying device capacities.

Despite its benefits, aggregating independent low-rank matrices from clients introduces a critical issue known as *LoRA Aggregation Discordance* [33]. When standard federated averaging (FedAvg) is applied separately to the client-specific matrices $A_i$ and $B_i$, it introduces a fundamental mathematical inconsistency:

$$\frac{1}{N}\sum_{i=1}^{N}(B_i A_i) \neq \left(\frac{1}{N}\sum_{i=1}^{N} B_i\right)\left(\frac{1}{N}\sum_{i=1}^{N} A_i\right) \tag{2.14}$$

This inequality demonstrates that independent averaging of $B_i$ and $A_i$ fails to reconstruct the true average of the weight updates due to the non-linear, bilinear nature of matrix multiplication. As a result, applying FedAvg directly to these factorized matrices leads to inaccurate global aggregation and severe model degradation.

To address this discordance, the literature proposes several alternative strategies [33]. One approach is full-size matrix aggregation, where clients reconstruct the full $\Delta W = \frac{\alpha}{r} BA$ before transmission; however, this entirely negates the spatial communication benefits of LoRA. Another approach involves SVD recalculation, where the server aggregates full matrices and then performs Singular Value Decomposition to extract new low-rank factors, which introduces prohibitive computational overhead at the server. Compared to these computationally or communication-heavy alternatives, alternating single-matrix aggregation has emerged as a highly efficient and structurally sound solution.

## 2.7 Robust Low-Rank Adaptation

While LoRA successfully reduces the spatial communication cost, the aggregation inconsistency described above poses a fundamental barrier to its deployment in federated settings. RoLoRA addresses this without sacrificing adapter expressiveness.

To resolve this inconsistency without limiting adapter expressiveness, RoLoRA [6] introduces an alternating optimization and aggregation framework. Instead of permanently freezing one of the low-rank matrices, RoLoRA alternates the active training coordinate space across communication rounds.

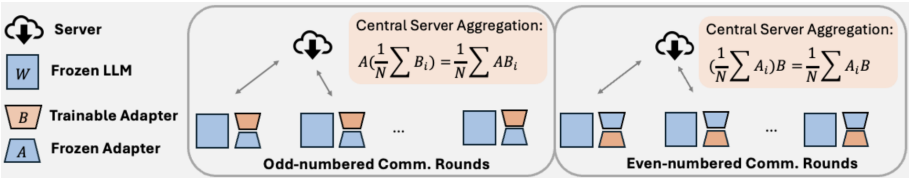



Figure 2.4: Robust Low-Rank Adaptation (RoLoRA) alternating optimization scheme. In each round, one low-rank matrix is frozen globally while the other is trained and aggregated linearly. *Note: The schematic reproduced from Chen et al. [6] uses row-vector notation ($AB$); under our standard column-vector convention ($BA$), odd rounds aggregate $(\frac{1}{N}\sum B_i)A$ and even rounds aggregate $B(\frac{1}{N}\sum A_i)$.*

*Note on notation:* While Chen et al. [6] designate their low-rank product as $AB$ by naming the up-projection $A$ and down-projection $B$, we maintain the standard LoRA convention $\Delta W = \frac{\alpha}{r}BA$ [11] throughout this thesis to preserve mathematical consistency with common PEFT architectures and the formal framework developed in Chapter 3.

In odd-numbered communication rounds $(2t+1)$, the down-projection matrix $A^t$ is globally frozen and shared across all clients $i$, while only the up-projection matrix $B$ is trained locally. The server then aggregates these updates:

$$B^{t+1} = \frac{1}{N}\sum_{i=1}^{N} B_i^{t+1} \tag{2.15}$$

Because $A^t$ is identical across all clients, the aggregation is strictly linear, introducing no spatial interference:

$$\frac{1}{N}\sum_{i=1}^{N}(B_i^{t+1}A^t) = \left(\frac{1}{N}\sum_{i=1}^{N} B_i^{t+1}\right)A^t = B^{t+1}A^t \tag{2.16}$$

In even-numbered communication rounds $(2t+2)$, the aggregated up-projection matrix $B^{t+1}$ is frozen, while the clients update the down-projection matrix $A$ locally. The server aggregates these updates:

$$A^{t+1} = \frac{1}{N}\sum_{i=1}^{N} A_i^{t+1} \tag{2.17}$$

Since $B^{t+1}$ is now identical across all clients, linearity is preserved:

$$\frac{1}{N}\sum_{i=1}^{N}(B^{t+1}A_i^{t+1}) = B^{t+1}\left(\frac{1}{N}\sum_{i=1}^{N} A_i^{t+1}\right) = B^{t+1}A^{t+1} \tag{2.18}$$

This alternating optimization enables full parameter space exploration while maintaining zero-interference linear aggregations throughout the federated training process.

## 2.8 Federated Dynamic Averaging

Having resolved the spatial communication bottleneck through RoLoRA's linearized aggregation, we now address the temporal dimension: controlling the synchronization frequency based on actual training dynamics rather than a fixed schedule.

Federated Dynamic Averaging (FDA) and its optimization variant FDA-Opt [28] were developed to optimize the training schedule dynamically. FDA is rooted in the principles of Geometric Monitoring [25], a framework originally developed for threshold function monitoring over distributed data streams.

### 2.8.1 Geometric Monitoring Foundations

The core insight of geometric monitoring is that a global condition over distributed data can be decomposed into local conditions that each node monitors independently. Consider $N$ distributed nodes, each holding a local state vector $v_i(t)$ that evolves over time. The global state is the average $\bar{v}(t) = \frac{1}{N}\sum_{i=1}^{N} v_i(t)$. The monitoring objective is to detect when a threshold function $f(\bar{v}(t))$ crosses a predefined boundary, without requiring continuous global synchronization.

Geometric monitoring achieves this through the concept of *safe zones*. Each node $i$ defines a bounding region around a reference point $e_i$ (typically the last synchronized global estimate). As long as the local drift vector $\delta_i(t) = v_i(t) - e_i$ remains within its safe zone, the global condition is guaranteed to hold by a convexity argument: if all local states lie within a convex safe zone centered around the global reference, their average must also lie within that zone. Crucially, this means that *no communication is necessary* as long as every node's local constraint is satisfied. A synchronization round is triggered only when at least one node's drift exits its safe zone, indicating that the global condition may have been violated.

FDA adapts this distributed monitoring framework to the federated learning setting. The "state vectors" become local model parameter updates, the "threshold function" becomes model variance (measuring client drift divergence), and the "safe zone" is defined by a variance budget $\theta$. When local parameter drift across clients causes the variance to exceed $\theta$, synchronization is triggered. This reformulation transforms the rigid, schedule-based synchronization of standard FL into an event-driven protocol that communicates only when training dynamics demand it.

For a sampled subset of clients $S_t$ at communication round $t$, let $\Delta_{t,i}^{(k)} = w_{t,i}^{(k)} - w_{t,0}$ denote the local update of client $k$ at step $i$ relative to the initial global model $w_{t,0}$. The average update (pseudo-gradient) across all clients is defined as:

$$g_{t,i} = \frac{1}{|S_t|} \sum_{k \in S_t} \Delta_{t,i}^{(k)} \tag{2.19}$$

The model variance quantifies the dispersion of local client updates in the parameter space and is defined as:

$$\mathrm{Var}(W_{t,i}) = \frac{1}{|S_t|} \sum_{k \in S_t} \left\| \Delta_{t,i}^{(k)} \right\|_2^2 - \|g_{t,i}\|_2^2 \tag{2.20}$$

Here, the first term measures the average local training progress across clients, while the second term captures the magnitude of the aggregated global step. Consequently, a low model variance value signifies either negligible local gradient progress or, more favorably, highly aligned local updates across participating clients where local optimization trajectories coincide. Conversely, a high variance reading manifests when clients achieve substantial local parameter movement along divergent optimization vectors towards conflicting local minima, signaling the onset of severe client drift.

During local training, clients track model variance. Computing this exact variance requires full-dimensional vector operations, which can be computationally expensive and communication-heavy to track continuously in a distributed setting. To address this, the original FDA protocol utilizes the Alon-Matias-Szegedy (AMS) sketch [2], a randomized algorithm that efficiently approximates the $L_2$ norm (and thus the variance) using minimal memory and sub-linear space. While full-parameter models require randomized sketching to approximate variance in sub-linear memory, we show in Chapter 3 that restricting geometric monitoring to compact low-rank coordinate tensors allows exact variance evaluation with negligible computational and memory overhead, rendering probabilistic sketching unnecessary in federated LoRA regimes.

Synchronization is triggered immediately if the variance exceeds a dynamic threshold: $\mathrm{Var}(W_{t,i}) > \theta_t$. Because the variance magnitude evolves by orders of magnitude from early to late training stages, FDA-Opt dynamically scales this threshold across rounds using a linear prediction based on the previous round's termination step $s_t$:

$$\theta_{t+1} = \mathrm{var}_t \cdot \frac{\tau}{2s_t} \tag{2.21}$$

where $\mathrm{var}_t$ is the actual variance at termination in round $t$, $s_t$ is the termination step, and $\tau$ is the target training duration. This linear threshold prediction assumes that standard training variance typically grows linearly and monotonically during a round. Furthermore, to eliminate the synchronization overhead of checking variance at every training step, queries are restricted to a sparse subset of steps (typically once per epoch), mitigating the communication bottleneck of distributed geometric checks.

While RoLoRA resolves the spatial aggregation inconsistency and FDA addresses the temporal communication cadence, combining the two paradigms requires careful algorithmic formulation. In particular, low-rank parameter trajectories exhibit heightened variance volatility, and alternating optimization demands decoupled threshold management. To address these challenges, Chapter 3 formalizes **FDA-RoLoRA** to combine dynamic averaging with alternating low-rank adaptation, explaining how it internally deploys a dual-track threshold mechanism to track subspace divergence independently. The concrete execution loops of both baseline protocols are formally summarized in Section 3.2 before introducing our synthesized methodology.

# Chapter 3

# Methodology

The research presented in this thesis investigates the integration of Federated Dynamic Averaging (FDA) [28] with Parameter-Efficient Low-Rank Adaptation (LoRA) [11]. By bridging dynamic variance-driven synchronization with low-rank parameter factorizations, this work aims to jointly optimize the spatial and temporal dimensions of federated learning.

## 3.1 Combining Dynamic Averaging with Federated LoRA

Existing literature in Parameter-Efficient Federated Fine-Tuning focuses almost exclusively on spatial parameter compression, while universally relying on static Federated Averaging (FedAvg) [21] for temporal synchronization. Under standard FedAvg, all clients execute a predetermined, fixed number of local epochs ($\tau = E$) regardless of local optimization drift, data heterogeneity, or gradient volatility.

The concept of integrating dynamic averaging into federated low-rank architectures envisions replacing rigid, static FedAvg schedules with adaptive, variance-driven averaging. Crucially, combining FDA with different federated LoRA variants cannot be treated as a single, one-size-fits-all formula. Because distinct PEFT methods manipulate low-rank matrices through fundamentally different mechanisms (ranging from simultaneous factor updates to frozen projections, sparse allocations, or alternating coordinates), each pairing suggests a distinct architectural design:

- **FDA + FedLoRA**: Explores dynamic synchronization alongside standard simultaneous training of low-rank matrices $A$ and $B$. In this configuration, dynamic averaging could monitor the joint parametric drift or product-space variance of the complete adapter. While this pairing retains the characteristic bilinear aggregation of standard FedLoRA ($\frac{1}{N}\sum_{i=1}^{N}(B_iA_i) \neq \left(\frac{1}{N}\sum_{i=1}^{N}B_i\right)\left(\frac{1}{N}\sum_{i=1}^{N}A_i\right)$), it represents the most direct, unconstrained synthesis of dynamic scheduling and low-rank adaptation.

- **FDA + FFA-LoRA**: Explores dynamic averaging under frozen down-projections [27], where matrix $A$ remains fixed after random initialization. Because parameter evolution is restricted solely to the up-projection $B$, dynamic variance monitoring simplifies to tracking a single active subspace coordinate space.

- **FDA + SLoRA**: Explores dynamic synchronization tailored to sparse low-rank parameter representations [3], where threshold adaptation and variance estimation could be structured around coordinate-sparse adapter allocations across clients.

- **FDA + RoLoRA (Our investigation)**: Explores dynamic averaging in conjunction with alternating coordinate optimization [6]. By alternating active updates between down-projection matrix $A$ and up-projection matrix $B$ across global communication rounds, this avenue naturally isolates each subspace, preserving update linearity ($\frac{1}{N}\sum_{i=1}^{N}(B_iA) = \left(\frac{1}{N}\sum_{i=1}^{N}B_i\right)A$) and enabling independent, decoupled threshold tracking ($\theta_A, \theta_B$).

Among these architectural possibilities, we select **FDA + RoLoRA** as our focal investigation due to the unique mathematical advantages of coordinate alternation. By isolating updates to a single low-rank factor per communication round, RoLoRA provides strict linearity in parameter aggregation, eliminating inter-factor interference. This structural property provides a clean theoretical foundation for formulating decoupled subspace variance tracking with independent geometric thresholds. We consider the broader taxonomy (such as joint variance tracking in simultaneous FedLoRA or coordinate-sparse thresholding in SLoRA) as promising avenues for future research.

The key insight enabling this integration is that FDA's variance monitoring can be applied independently to each subspace, as RoLoRA's alternating scheme naturally isolates the optimization trajectories of the down-projection ($A$) and up-projection ($B$) matrices. To regulate threshold updates across communication rounds, the framework adopts a generalized proportional setpoint scaling rule ($k \cdot e_{max}$ from Eq. 3.9, where $k = 0.5$ recovers the midpoint and $k = 1.0$ targets the upper bound) that dynamically adapts synchronization intervals to local client drift.

## 3.2 Foundational Precursor Protocols: RoLoRA and FDA

To rigorously contextualize the design of FDA-RoLoRA, we examine the operational execution loops of its two constituent foundational frameworks: Federated Dynamic Averaging (FDA) [28] and Robust Low-Rank Adaptation (RoLoRA) [6]. While both methods address fundamental inefficiencies in decentralized learning, they tackle orthogonal dimensions of the federated optimization bottleneck:

- **FDA (Temporal Optimization, Full-Rank Model Space)**: Evaluates the dispersion of local client updates in parameter space. By deploying geometric safe zones via the AMS sketch, FDA replaces static round schedules with variance-triggered synchronization, dynamically terminating local training at $e^* \leq E$ when client drift exceeds a scalar threshold $\theta$. However, the standard protocol assumes simultaneous parameter updates over the full model weight space, leaving low-rank factorized architectures unaddressed.

- **RoLoRA (Spatial Optimization, Fixed Local Schedule)**: Eliminates bilinear aggregation inconsistency by alternating coordinate optimization between low-rank factor matrices ($A$ and $B$) across global communication rounds. When factor $B$ is trained, down-projection $A$ is frozen, preserving update linearity ($\frac{1}{N}\sum_{i=1}^{N}(B_iA) =$

$\left(\frac{1}{N}\sum_{i=1}^{N} B_i\right) A$). However, RoLoRA enforces a rigid, fixed local epoch schedule ($E$) across all clients and rounds, leaving temporal synchronization unoptimized.

Algorithm 1 and Algorithm 2 summarize the algorithmic structures of FDA and RoLoRA side-by-side.

**Algorithm 1:** FDA

**Input:** Clients $K$, cohort $|S_r|$, initial $\theta_0$, limit $E$

Initialize global parameters $W_0$;
**for** *round* $r = 0, 1, 2, \ldots$ **do**
  Broadcast $W_0^{(r)}$ to cohort $S_r$;
  **for** *epoch* $e = 1$ **to** $E$ **do**
    **foreach** *client* $i \in S_r$ ***in parallel*** **do**
      $W_{i,e}^{(r)} \leftarrow W_{i,e-1}^{(r)} - \eta_c \nabla \mathcal{L}_i$;
      $\Delta_i^{(e)} = W_{i,e}^{(r)} - W_0^{(r)}$;
    Evaluate AMS variance;
    **if** $Var(\Delta^{(e)}) > \theta_r$ **then**
      $e^* \leftarrow e$; **break**;
  $W_0^{(r+1)} \leftarrow \frac{1}{|S_r|}\sum_{i \in S_r} W_{i,e^*}^{(r)}$;
  Update scalar threshold $\theta_{r+1}$;

**Algorithm 2:** RoLoRA

**Input:** Clients $K$, cohort $|S_r|$, fixed epoch limit $E$

Initialize $W = W_{pretrained} + \frac{\alpha}{r} B_0 A_0$;
**for** *round* $r = 0, 1, 2, \ldots$ **do**
  Select active $\mathcal{M}^{(r)} \in \{B, A\}$;
  Freeze complementary matrix;
  Broadcast active $W_{act}^{(r)}$ to $S_r$;
  **for** *epoch* $e = 1$ **to** $E$ **do**
    **foreach** *client* $i \in S_r$ ***in parallel*** **do**
      Train $\mathcal{M}^{(r)}$ on local data;
      $\Delta_i = W_{i,e}^{(r)} - W_{act}^{(r)}$;
  Collect client drifts $\{\Delta_i\}_{i \in S_r}$;
  $\Delta_{agg} \leftarrow \frac{1}{|S_r|}\sum_{i \in S_r} \Delta_i$;
  $W_{act}^{(r+1)} \leftarrow W_{act}^{(r)} + \Delta_{agg}$;

Contrasting these two protocols reveals the central architectural gap: RoLoRA provides the necessary update linearity to eliminate spatial distortion, but lacks adaptive temporal control; FDA provides dynamic temporal adaptivity, but cannot handle coordinate-isolated, asymmetric low-rank subspaces with a single scalar threshold. This limitation directly motivates our proposed **FDA-RoLoRA**, which synthesizes alternating coordinate isolation with decoupled subspace variance tracking via an internal dual-track threshold mechanism.

## 3.3 Notation and Hyperparameters

Consider a global communication round indexed by $r$, during which an active matrix $\mathcal{M}^{(r)} \in \{A, B\}$ is designated for client-side optimization. Within round $r$, local client optimization progresses through discrete local epoch iterations $e \in \mathbb{N}$, culminating in a final realized local epoch count $e^* \in \mathbb{N}$ at the instant synchronization is triggered. At any given evaluation epoch $e$, each client $i$ computes its raw subspace parametric drift $\Delta_i^{(e)}$ relative to the starting global parameters. Rather than projecting these parameter updates back to the expanded model weight space, the framework evaluates the direct subspace variance $\mathrm{Var}_{subspace}(\Delta^{(e)})$ exclusively on the low-rank coordinate tensors of the active matrix.

The operational boundaries and adaptive behavior of FDA-RoLoRA are governed by a set of global hyperparameters. Local client training is bounded between an operational minimum epoch floor $e_{min} \in \mathbb{N}$ and a maximum epoch ceiling $e_{max} \in \mathbb{N}$. In practice, $e_{min}$ serves as an operational lower bound (for instance, setting $e_{min} = 1$) to ensure clients execute an initial computation pass before variance evaluations begin, though the framework does not presuppose that a particular value is universally fixed or optimal. To accommodate the distinct drift dynamics of alternating low-rank factor matrices, FDA-RoLoRA internally implements a dual-track threshold mechanism: synchronization timing is controlled by two persistent, decoupled variance thresholds $\theta_A, \theta_B \in \mathbb{R}^+$, from which the active threshold $\theta_{current}^{(r)}$ is dynamically mapped to match $\mathcal{M}^{(r)}$. Threshold adaptation across rounds is governed by variance-scaled setpoint feedback. A comprehensive summary of mathematical symbols, tensor shapes, and hyperparameters is provided in Table 3.1.

Table 3.1: Summary of mathematical notation and system hyperparameters.

| Symbol | Domain / Dimensions | Description |
|---|---|---|
| $W_{pretrained}$ | $\mathbb{R}^{d_{out} \times d_{in}}$ | Frozen backbone model parameter weights |
| $r_{rank}$ | $\mathbb{N}$ ($r_{rank} \ll \min(d_{in}, d_{out})$) | Intrinsic low-rank adapter rank dimension |
| $A$ | $\mathbb{R}^{r_{rank} \times d_{in}}$ | Low-rank down-projection factor matrix |
| $B$ | $\mathbb{R}^{d_{out} \times r_{rank}}$ | Low-rank up-projection factor matrix |
| $K$ | $\mathbb{N}^+$ | Total number of decentralized client nodes in the federation |
| $S_r$ | $S_r \subseteq \{1, \dots, K\}$ | Sampled cohort of participating clients in round $r$ |
| $r$ | $\mathbb{N}_0 = \{0, 1, 2, \dots\}$ | Global communication round index |
| $\mathcal{M}^{(r)}$ | $\{B, A\}$ | Designated active parameter matrix in round $r$ |
| $W_{act}^{(r)}$ | Active matrix tensor | Global parameters of the active matrix broadcast to cohort $S_r$ |
| $e$ | $\{1, \dots, e_{max}\}$ | Local training epoch index within a round |
| $e_{min}, e_{max}$ | $\mathbb{N}^+$ ($e_{min} \leq e_{max}$) | Operational minimum epoch floor (e.g., $e_{min} = 1$) and maximum epoch ceiling |
| $e^*$ | $[e_{min}, e_{max}]$ | Realized local epoch count at synchronization trigger |
| $\Delta_i^{(e)}$ | Matching $\mathcal{M}^{(r)}$ | Subspace parametric drift of client $i$ at epoch $e$ |
| $\mathrm{Var}_{subspace}$ | $\mathbb{R}_{\geq 0}$ | Subspace drift variance across cohort $S_r$ (Eq. 3.5) |
| $\theta_A, \theta_B$ | $\mathbb{R}^+$ | Persistent, decoupled variance thresholds for tracks $A$ and $B$ |
| $\theta_{current}^{(r)}$ | $\mathbb{R}^+$ | Active threshold loaded for round $r$ ($\theta_A$ or $\theta_B$) |
| $k$ | $(0, 1]$ | Proportional controller setpoint ratio (e.g., $k = 0.5$ for $e_{max}/2$, $k = 1.0$ for $e_{max}$) |
| $\nabla_{pseudo}^{(r)}$ | Matching $\mathcal{M}^{(r)}$ | Aggregated server-side pseudo-gradient (Eq. 3.7) |
| $\eta_c, \eta_s$ | $\mathbb{R}^+$ | Local client learning rate and global server learning rate |

**Algorithm 3:** FDA-RoLoRA

**Input:** Clients $K$, cohort size $|S_r|$, server learning rate $\eta_s$, local learning rate $\eta_c$, min/max epochs $e_{min}, e_{max}$, setpoint ratio $k$, initial thresholds $\theta_A, \theta_B$

Initialize global parameters $W = W_{pretrained} + \frac{\alpha}{r} B_0 A_0$;
Initialize persistent thresholds $\theta_A, \theta_B$ and decoupled server optimizer states (for matrices $A$ and $B$);
**for** *round* $r = 0, 1, 2, \ldots$ **do**
    Determine active matrix $\mathcal{M}^{(r)} \in \{B, A\}$ (alternate $B$-first each round);
    Load active threshold $\theta^{(r)}_{current} \leftarrow \theta_{\mathcal{M}^{(r)}}$ and active server optimizer;
    Sample client cohort $S_r \subseteq \{1, \ldots, K\}$;
    Server broadcasts active parameters $W^{(r)}_{act}$ to sampled cohort $S_r$;
    **for** *local epoch* $e = 1$ **to** $e_{max}$ **do**
        **foreach** *client* $i \in S_r$ ***in parallel*** **do**
            Train locally for one epoch on private data: $W^{(r)}_{i,e} \leftarrow W^{(r)}_{i,e-1} - \eta_c \nabla \mathcal{L}_i(W^{(r)}_{i,e-1})$;
            Compute subspace drift: $\Delta^{(e)}_i = W^{(r)}_{i,e} - W^{(r)}_{act}$;
        **if** $e \geq e_{min}$ **then**
            Server collects drifts $\{\Delta^{(e)}_i\}_{i \in S_r}$ and evaluates cohort variance $\mathrm{Var}_{subspace}(\Delta^{(e)})$ via Eq. 3.5;
            **if** $Var_{subspace}(\Delta^{(e)}) > \theta^{(r)}_{current}$ ***or*** $e = e_{max}$ **then**
                $e^* \leftarrow e$;
                **break**;
    Server computes pseudo-gradient $\nabla^{(r)}_{pseudo} = -\frac{1}{|S_r|} \sum_{i \in S_r} \Delta^{(e^*)}_i$ via Eq. 3.7;
    Update active matrix threshold $\theta_{\mathcal{M}^{(r)}} \leftarrow \left(\frac{k \cdot e_{max}}{e^*}\right) \mathrm{Var}_{subspace}(\Delta^{(e^*)})$ via Eq. 3.9, keeping inactive threshold frozen;
    Apply active server optimizer (Gradient Descent / FedAvg with $\eta_s$) to update $W^{(r+1)}_{act}$ via Eq. 3.10;

## 3.4 FDA-RoLoRA Initialization and Subspace Toggling

At the start of round $r$, the server determines the active target matrix $\mathcal{M}^{(r)} \in \{B, A\}$ adhering to a $B$-first alternating schedule. Internally, FDA-RoLoRA operates a dual-track threshold mechanism that maintains independent variance tracking for each coordinate space. To govern the local synchronization dynamics, the system loads the persistent threshold corresponding exclusively to the active matrix:

$$\theta_{current}^{(r)} = \begin{cases} \theta_A & \text{if } \mathcal{M}^{(r)} = A \\ \theta_B & \text{if } \mathcal{M}^{(r)} = B \end{cases} \tag{3.1}$$

The active weight matrix is initialized across all sampled clients $i \in S_r$ using the global model parameters broadcast by the server:

$$W_{i,0}^{(r)} = W_{act}^{(r)} \tag{3.2}$$

## 3.5 Local Training and Direct Subspace Variance

Clients advance local parameters using independent Stochastic Gradient Descent [24] steps. For client $i$ at local step $t$, the parameter update operates on the active parameter tensor:

$$W_{i,t}^{(r)} = W_{i,t-1}^{(r)} - \eta_c \nabla \mathcal{L}_i(W_{i,t-1}^{(r)}) \tag{3.3}$$

where $\eta_c$ is the local learning rate and $\mathcal{L}_i$ is the client's empirical loss.

Local training proceeds epoch-by-epoch. At each epoch boundary $e \geq e_{min}$, participating clients compute their raw subspace parametric drift relative to the starting global parameter state:

$$\Delta_i^{(e)} = W_{i,e}^{(r)} - W_{act}^{(r)} \tag{3.4}$$

To minimize communication overhead, exact variance is computed directly on the isolated low-rank coordinates of the active matrix, avoiding projection to the expanded parameter space:

$$\mathrm{Var}_{subspace}(\Delta^{(e)}) = \frac{1}{|S_r|} \sum_{i \in S_r} \left\| \Delta_i^{(e)} \right\|_2^2 - \left\| \frac{1}{|S_r|} \sum_{i \in S_r} \Delta_i^{(e)} \right\|_2^2 \tag{3.5}$$

Crucially, performing this variance check at intermediate epoch boundaries ($e \in [e_{min}, e_{max}]$) does not impose a prohibitive communication burden on the federated network. In contemporary foundation models (such as RoBERTa-Large or Llama-2-7B), low-rank adapter factor matrices ($r_{rank} = 4$, or more generally $r_{rank} \in \{4, 8\}$) represent less than 1% (and frequently under 0.2%) of the total backbone parameter footprint, yielding a greater than 99% parameter reduction relative to full-rank fine-tuning. Mathematically, evaluating cohort subspace variance $\mathrm{Var}_{subspace}(\Delta^{(e)}) = \frac{1}{|S_r|} \sum_i \|\Delta_i^{(e)}\|_2^2 - \|\frac{1}{|S_r|} \sum_i \Delta_i^{(e)}\|_2^2$ requires calculating the norm of the average cohort drift, which cannot be computed locally from scalar norms alone. In our simulation testbed, exact low-rank variance is evaluated directly in memory across intermediate epoch boundaries. In a bandwidth-constrained physical deployment, clients transmit lightweight compressed AMS sketches (6 KB payload) at intermediate epoch boundaries to evaluate drift variance with provable geometric error bounds,

adding only 2.5%–5.8% communication overhead across training rounds, while the complete adapter tensor (120 KB uplink for 5 layers, or 72 KB for 3 layers) is transmitted only once per round upon synchronization halt at $e^*$.

## 3.6 Server-Coordinated Synchronization Halt

At each epoch boundary $e \geq e_{min}$, the server evaluates the collective cohort variance $\mathrm{Var}_{subspace}(\Delta^{(e)})$ (computed directly in memory in our simulation testbed, or triggered via lightweight scalar norm and geometric safe-zone checks in physical deployments so that the 120 KB low-rank uplink tensor is transmitted only once upon synchronization halt at $e^*$). The server then determines whether to trigger global aggregation or instruct clients to proceed to the subsequent local epoch:

$$\text{Halt Local Training and Trigger Aggregation if } \mathrm{Var}_{subspace}(\Delta^{(e)}) > \theta^{(r)}_{current} \quad \vee \quad e \geq e_{max} \tag{3.6}$$

If the drift variance remains within the threshold and $e < e_{max}$, clients continue local optimization without model aggregation. The final realized local epoch count at synchronization is recorded as $e^* \in [e_{min}, e_{max}]$.

As established in Section 4.1, the operational floor $e_{min}$ prevents premature synchronization before initial gradient steps accumulate, while preserving dynamic variance evaluation across $[e_{min}, e_{max}]$.

## 3.7 Global Aggregation and Independent Threshold Evolution

Upon receiving the client updates, the server aggregates the subspace drifts to compute the pseudo-gradient:

$$\nabla^{(r)}_{pseudo} = -\frac{1}{|S_r|} \sum_{i \in S_r} \Delta_i^{(e^*)} \tag{3.7}$$

The server then updates the active matrix parameters. Crucially, the variance threshold for the active matrix $\mathcal{M}^{(r)}$ is updated independently to preserve state for its next training cycle, while the threshold for the inactive matrix remains frozen.

### 3.7.1 Foundational Midpoint Scaling from FDA-Opt

We initially formulate the threshold adaptation by directly inheriting the canonical midpoint-scaling update established in the foundational full-rank FDA-Opt framework [28]. Under this baseline rule, the active threshold adjusts dynamically based on the realized epoch count $e^*$ relative to the nominal training midpoint $e_{max}/2$:

$$\theta^{(next)}_{\mathcal{M}^{(r)}} = \left( \frac{e_{max}/2}{e^*} \right) \mathrm{Var}_{subspace}(\Delta^{(e^*)}) \tag{3.8}$$

This direct scaling establishes a self-stabilizing negative feedback loop:

- When client updates diverge rapidly due to gradient volatility or data heterogeneity, the variance threshold is breached early ($e^* < e_{max}/2$). Consequently, the scaling

multiplier $(e_{max}/2)/e^* > 1$ expands the threshold for the subsequent round of that coordinate track, preventing premature synchronization halts.

- Conversely, when client trajectories remain aligned and stable ($e^* > e_{max}/2$), the scaling multiplier is strictly less than 1, systematically tightening the threshold to encourage timely synchronization before excessive local client drift accumulates.

### 3.7.2 From Canonical Midpoint to Tunable Epoch Setpoints

While the canonical $e_{max}/2$ midpoint rule provides clean negative feedback, empirical analysis across disparate data distributions revealed that targeting a fixed midpoint does not behave uniformly across federated regimes:

- **Non-IID Regimes**: Under substantial statistical heterogeneity (such as severe Dirichlet label skew), client parameter trajectories diverge rapidly. In this setting, the $e_{max}/2$ midpoint provides an effective braking setpoint, successfully preventing catastrophic local client drift.

- **IID Regimes**: Conversely, when client data distributions are homogeneous (IID), inter-client variance is naturally minimal because clients optimize along near-consensus gradients. Under these conditions, client updates remain well-behaved across extended training horizons. Enforcing a hardwired target of $e_{max}/2$ causes the threshold to contract excessively, prematurely truncating training when clients could have safely executed further local epochs to minimize communication rounds.

To accommodate both homogeneous consensus and heterogeneous dispersion, we generalize the threshold scaling update by introducing a proportional controller setpoint ratio $k \in (0, 1]$:

$$\theta_{\mathcal{M}^{(r)}}^{(next)} = \left( \frac{k \cdot e_{max}}{e^*} \right) \mathrm{Var}_{subspace}(\Delta^{(e^*)}) \tag{3.9}$$

Under this generalized formulation, setting $k = 0.5$ exactly recovers the canonical FDA-Opt midpoint rule ($0.5e_{max} = e_{max}/2$). In contrast, an aggressive setpoint such as $k = 1.0$ targets the upper epoch ceiling ($e^* \approx e_{max}$), allowing federated clients to maximize local computation when drift variance remains subdued. Intermediate setpoints (such as $k = 0.75$, targeting $\frac{3}{4}e_{max}$) provide balanced trade-offs.

In our methodological formulation, Eq. 3.8 represents the foundational midpoint rule inherited from previous work, while Eq. 3.9 provides the parametric control law. In Chapter 5, we empirically evaluate the behavioral impact of transitioning from canonical midpoint scaling ($k = 0.5$) to full-horizon scaling ($k = 1.0$) across both IID and Non-IID benchmark suites.

## 3.8 Server Optimization and Aggregation Dynamics

Following local client execution, the server aggregates client progress to perform the global model update. In the precursor FDA-Opt framework [28], aggregated client parameter displacements were formulated as pseudo-gradients $\nabla_{pseudo}^{(r)} = -\frac{1}{|S_r|} \sum_{i \in S_r} \Delta_i^{(e^*)}$ (Eq. 3.7) to interface with adaptive server-side optimizers (such as FedAdam or FedYogi). We carried this pseudo-gradient interface over into the FDA-RoLoRA codebase to preserve modular extensibility for future adaptive optimization research.

In our production benchmark experiments, however, the server update is configured strictly with standard Gradient Descent (SGD) with server learning rate $\eta_s = 1.0$:

$$W_{act}^{(r+1)} = W_{act}^{(r)} - \eta_s \nabla_{pseudo}^{(r)} = W_{act}^{(r)} + \frac{1}{|S_r|} \sum_{i \in S_r} \Delta_i^{(e^*)} = \frac{1}{|S_r|} \sum_{i \in S_r} W_{i,e^*}^{(r)} \tag{3.10}$$

Setting $\eta_s = 1.0$ collapses the server update directly to coordinate-wise Federated Averaging (FedAvg) over the active low-rank factor matrix. Adopting direct linear aggregation ensures that all empirical gains observed in this thesis stem directly from the dynamic, variance-driven synchronization mechanism rather than confounding server-side momentum or second-moment preconditioning artifacts.

## 3.9 Architectural Exploration and Evolutionary Design Decisions

The finalized architecture of FDA-RoLoRA is the outcome of systematically exploring and discarding several preliminary formulations during pilot development. Documenting these rejected architectural paths provides critical insight into the structural challenges of combining dynamic federated averaging with low-rank factorizations:

### 3.9.1 Dense Product Reconstruction and SVD Factorization

An intuitive initial approach to federated low-rank adaptation involves allowing clients to optimize both low-rank factor matrices ($A_i$ and $B_i$) simultaneously, transmitting their parameter updates to the server to compute the accumulated dense parameter drift $\Delta W_i = B_i A_i$. Under this protocol (*FDA-LoRA-SVD*), the server aggregates the dense client updates in full model weight space, applies a global optimizer step, and subsequently performs truncated Singular Value Decomposition (SVD):

$$W^{(r+1)} \approx U_r \Sigma_r V_r^T$$

re-factoring the principal components back into rank-$r$ matrices $B \leftarrow U_r \Sigma_r^{1/2}$ and $A \leftarrow \Sigma_r^{1/2} V_r^T$.

While mathematically unconstrained, pilot profiling revealed that this approach is computationally prohibitive. Evaluating exact matrix drift requires reconstructing full dense matrices ($d_{\text{out}} \times d_{\text{in}}$) in memory across all clients at each epoch boundary, inflating GPU VRAM footprint and eliminating the memory benefits of PEFT. Furthermore, transmitting and aggregating full dense matrices across clients scales with $\mathcal{O}(d^2)$ communication and memory, completely eliminating the bandwidth advantages of PEFT. Factoring the aggregated dense updates back into rank-$r$ factors via truncated SVD requires an additional $\mathcal{O}(d^2 r)$ computational cost per layer at every round. These bottlenecks confirm that preserving factorized representation natively throughout training and communication is essential, directly motivating alternating coordinate optimization.

### 3.9.2 Single Shared Threshold vs. Decoupled Dual-Track

In early implementations of alternating coordinate optimization, local synchronization was governed by a single, shared scalar variance threshold $\theta$. However, empirical pilot

testing demonstrated that sharing a single threshold across alternating coordinate cycles induced acute optimization instability.

Because down-projection matrix $A$ and up-projection matrix $B$ play asymmetric roles (mapping input representations down to low-rank manifolds and projecting them back to model dimension, respectively), their parameter trajectories exhibit fundamentally different gradient magnitudes and geometric curvature. Under a single shared threshold, updates to matrix $B$ systematically distorted the threshold baseline loaded during matrix $A$'s subsequent activation round. This cross-matrix scale contamination caused erratic synchronization intervals, characterized by premature halting on one factor matrix and over-training on the other. Decoupling the threshold state into two strictly independent, persistent tracks $(\theta_A, \theta_B)$ completely eliminated this interference, yielding steady and monotonic convergence.

### 3.9.3 Phase Lag in Threshold Smoothing (EMA and SMA)

To reduce stochastic noise in threshold proposals between rounds, initial exploratory variants incorporated historical smoothing mechanisms, evaluating Exponential Moving Average (*Dual-Track EMA*) and sliding-window Simple Moving Average (*Dual-Track SMA*) update rules:

$$\text{EMA:} \quad \theta_{\mathcal{M}}^{(r+1)} = \rho\theta_{\mathcal{M}}^{(r)} + (1-\rho)\theta_{\text{target}}^{(r)}$$

$$\text{SMA:} \quad \theta_{\mathcal{M}}^{(r+1)} = \frac{1}{W}\sum_{w=0}^{W-1}\theta_{\text{target}}^{(r-w)}$$

While smoothing operators are standard in stationary time-series, our pilot evaluations showed that they performed suboptimally (and frequently worse) in alternating federated regimes. Because active coordinate optimization toggles between disjoint parameter subspaces across rounds, client drift trajectories undergo rapid phase changes. Historical moving-average buffers preserved obsolete information from earlier optimization regimes, causing the threshold to **lag behind** the actual drift velocity of the currently active matrix. This phase lag led to delayed synchronization responses, demonstrating that instantaneous, memoryless proportional setpoint feedback is structurally superior for alternating low-rank regimes.

In summary, this finalized closed-loop formulation constitutes the proposed **FDA-RoLoRA** framework evaluated across all empirical benchmarks in Chapter 5. By decoupling threshold updates into persistent coordinate tracks and anchoring synchronization to empirical drift variance, FDA-RoLoRA provides robust, adaptive communication horizons without manual tuning. The following Implementation chapter details the software architecture, the integration of Hugging Face PEFT, and the practical realization of this theoretical model within the experimental testbed.

# Chapter 4

# Implementation

The software architecture developed for this thesis realizes the integration of Federated Dynamic Averaging with parameter-efficient fine-tuning via FDA-RoLoRA, utilizing an internal dual-track threshold mechanism to manage coordinate subspace variance, open-sourced under the **FDA-RoLoRA** repository. This chapter details the software architecture, the strategies for data partitioning across clients, the configuration of the targeted foundation models, the mechanics of the training pipeline, and the high-performance computing environment utilized for experimental evaluation.

## 4.1 Software Architecture

The framework is developed in Python, utilizing the PyTorch deep learning framework as the foundational library for automatic differentiation and tensor operations. The codebase is built as a direct extension of the FDA-Opt repository [28],[1] inheriting its robust core federated orchestration infrastructure. To implement the adaptation strategies, the system heavily relies on the Hugging Face Transformers library in conjunction with the Parameter-Efficient Fine-Tuning (PEFT) library, which has established itself as the research standard for LoRA-based methods. The complete executable software architecture developed for this thesis has been open-sourced as **FDA-RoLoRA**.[2]

A key practical constraint faced during this research was the absence of a publicly available code repository for RoLoRA [6], which only noted using the FederatedScope-LLM platform [15]. Given that we had direct access to the FDA-Opt codebase [28], the design choice was between porting FDA's dynamic variance monitoring and threshold evolution logic into the heavy, event-driven architecture of FederatedScope-LLM, or implementing RoLoRA's alternating optimization protocol within the existing FDA-Opt framework. We chose the latter approach, as implementing RoLoRA inside the lightweight FDA-Opt core was substantially more tractable and transparent. This decision afforded complete architectural visibility and direct control over low-rank tensor serialization and threshold updates, without the abstraction overhead of a third-party framework.

To manage the complexities of federated learning, the system follows a clear three-tier architecture comprising a Server Orchestrator, Client Nodes, and a Communication Interface. The Server Orchestrator assumes a centralized role; it maintains the global LoRA parameter matrices $A$ and $B$, tracks the current threshold states $\theta_A$ and $\theta_B$ for synchronization, manages server-side optimizer momentum buffers, and increments a global round

[1] https://github.com/michaeltheologitis/FDA-Opt
[2] https://github.com/Asterinos1/FDA-RoLoRA

counter responsible for the RoLoRA alternating optimization toggle. The Client Nodes represent the edge devices or data silos. During each communication round, a subset of sampled clients receives the active parameter block ($A$ or $B$), performs local SGD on their private datasets across dynamic local epoch intervals, and returns the accumulated parameter drift vectors ($\Delta_i$) to the server, which centrally evaluates cohort variance and orchestrates the aggregation step. Finally, the Communication Interface orchestrates the logical data exchange between clients and the server. In our simulation testbed, client-server communication is modeled in-silico: rather than incurring physical network socket overhead, parameter drift vectors ($\Delta_i$) are calculated via direct in-memory tensor subtractions against the round baseline snapshot. Communication bandwidth consumption is evaluated analytically by measuring the exact bi-directional serialized byte footprint of the active low-rank coordinate tensors transmitted per client per round (240 KB per client, consisting of 120 KB downlink broadcast and 120 KB uplink transmission under the 5-layer $\mathcal{P}_2$ configuration, totaling 4.6875 MB for a cohort of 20 clients in single-precision float32). Unlike conventional federated learning frameworks that transmit full model weights, this design yields a substantial reduction in bandwidth requirements.

Crucially, the alternating optimization protocol (the toggling mechanism inherent to the RoLoRA methodology) is strictly managed server-side. This centralization guarantees global consistency across all participating clients, ensuring that during any given round, every active client is optimizing and communicating updates for the same LoRA matrix block.

## 4.2 Data Partitioning

The software architecture implements client data partitioning following the Dirichlet allocation protocol defined in Section 2.4.4. During environment initialization, the partitioning module loads the targeted GLUE corpus. To address the significant disparity in scale across benchmark tasks (ranging from under 70,000 training examples in SST-2 to nearly 400,000 in MNLI) and mitigate prohibitive simulation runtimes on the shared GPU cluster, the framework adopts an empirical dataset ceiling. Specifically, the training volume is capped at an upper ceiling of 25,000 examples via deterministic pseudo-random subsampling, while smaller corpora and all validation evaluation sets are preserved in full. Preliminary pilot investigations confirmed that this sample budget provides sufficient linguistic diversity and gradient support for adapter fine-tuning without compromising convergence stability or task performance, while keeping multi-seed simulation horizons computationally tractable. Following this volume standardization, the module segments the active corpus into $K$ isolated local partitions according to the configured Dirichlet concentration parameter $\alpha$. Each simulated client receives a dedicated local partition and batch iterator, enforcing strict data isolation during local training without cross-client leakage.

## 4.3 Simulation Controls and Early Stopping

Beyond dataset scale, substantial wall-clock latency accumulates during simulation because client cohorts ($C \in \{20, 50\}$) are evaluated sequentially on the GPU to respect accelerator memory limits. To prevent excessive simulation runtimes across multi-seed sweeps and avoid computing redundant rounds on converged models, the framework in-

corporates an **accuracy-gated early stopping controller** into the Server Orchestrator. At the conclusion of each communication round, the server evaluates the consensus model on the centralized task validation split. The controller governs early termination via four complementary mechanisms:

1. **Trailing Sliding-Window Smoothing ($W$)**: In federated optimization under statistical heterogeneity, validation trajectories exhibit stochastic oscillations due to client sampling and non-IID drift. To prevent premature halts triggered by transient dips, the controller tracks a trailing moving average of validation accuracy across a sliding window of $W$ rounds.

2. **Accuracy Activation Gate ($A_{\text{gate}}$)**: Because alternating coordinate optimization requires several initial rounds to calibrate both factor matrices ($A$ and $B$), early stopping is strictly disarmed during early training. The patience counter activates only after the smoothed validation accuracy crosses a predefined milestone threshold ($A_{\text{gate}}$).

3. **Patience and Minimum Improvement ($\Delta_{\min}, P$)**: Once the activation gate is breached, the controller monitors validation progress over a patience window of $P$ consecutive rounds. If the moving-average accuracy fails to improve by at least $\Delta_{\min} = 0.5\%$, the training loop terminates cleanly ahead of $R_{\max}$.

4. **Mandatory Round Guard ($R_{\min}$)**: To ensure that models explore sufficient alternating coordinate cycles regardless of metric volatility, the engine enforces a strict round floor ($R_{\min}$) before any termination signal can trigger.

This controller functions as an external simulation management harness, accelerating cluster turnaround times without modifying the underlying federated optimization dynamics. The specific hyperparameter configurations deployed across each GLUE benchmark task and partitioning regime are detailed in Chapter 5 (Section 5.1).

## 4.4 Model Configuration

The experimental implementation targets transformer-based foundation models. In this work, the empirical evaluation focuses specifically on `roberta-base` [19] (comprising approximately 125 million parameters), evaluated on sequence classification tasks from the GLUE benchmark [31]. While the original RoLoRA paper [6] evaluated primarily on `roberta-large` (355M parameters), we deliberately chose `roberta-base` to accommodate our institutional hardware budget (2× NVIDIA A10 GPUs) and project turnaround time limits. This choice cuts VRAM allocation and backward pass latency by more than half, enabling thorough multi-seed sweeps across 20–50 clients while maintaining representative transformer attention geometry. Following the parameter budget $\mathcal{P}_2$ layer selection of RoLoRA [6] (which attaches adapters to top self-attention layers $\{15 \dots 23\}$ for MNLI, QNLI, and QQP, and $\{18 \dots 23\}$ for SST-2 on 24-layer `roberta-large`), we apply normalized depth mapping to the 12 encoder blocks (indices $0 \dots 11$) of `roberta-base`: low-rank adapters are injected into the query and value projection matrices ($W_q$ and $W_v$) of the top 5 encoder layers $\{7, 8, 9, 10, 11\}$ (10 active adapter matrices per round) for MNLI, QNLI, and QQP, and the top 3 encoder layers $\{9, 10, 11\}$ (6 active adapter matrices per round) for SST-2.

Across the model configuration, the LoRA rank is strictly constrained to $r = 4$ accompanied by scaling factor $\alpha_{\text{lora}} = 8$ to maintain activation magnitudes. During the federated fine-tuning process, all original pretrained foundation model weights as well as the downstream sequence classification head are kept completely frozen. The only trainable parameters across all communication rounds are the low-rank adapter matrices $A$ and $B$. This setup provides a substantial reduction in communication cost, as communicating only the compact low-rank drift matrices rather than the full 125M parameter set yields orders-of-magnitude reductions in client-server bandwidth requirements. While broader foundation models (such as RoBERTa-Large or autoregressive decoders like Llama-2-7B) represent natural targets for future scaled deployment, our implementation and empirical study focus on `roberta-base` across GLUE tasks.

## 4.5 Training Pipeline

The operational training pipeline translates the theoretical FDA-RoLoRA protocol (formulated in Algorithm 3) into an efficient, stateful PyTorch execution loop. At the start of communication round $t$, the Server Orchestrator determines the active parameter track by evaluating round parity ($t \pmod 2$), activating track $B$ on even rounds and track $A$ on odd rounds. To enforce alternating coordinate isolation in software, the orchestrator traverses the model parameter tree (`model.named_parameters()`) and dynamically toggles the PyTorch autograd flags: parameters matching the active adapter key are assigned `requires_grad = True`, while the dormant adapter block, the downstream classification head, and all backbone foundation model parameters are assigned `requires_grad = False`. The server snapshots the active parameters ($W_{\text{act}}^{(0)}$) as the baseline for tracking parametric drift, binds the corresponding independent server optimizer instance, loads the active threshold state ($\theta_B$ or $\theta_A$), and synchronizes the active weights to the sampled client cohort $S_t \subset \{1, \ldots, K\}$. Maintaining two persistent `torch.optim` instances across rounds keeps optimizer momentum states strictly partitioned, preventing cross-phase state contamination.

Within each simulated communication round, sampled clients are evaluated sequentially on the allocated GPU accelerator, executing local SGD over their private data partitions using a configurable local learning rate $\eta_c$ and local batch size $B$. This sequential simulation design faithfully models the logical decentralized execution of independent client nodes while strictly constraining GPU VRAM footprint to a single active client instance at any given instant. Because inactive weights have autograd disabled, PyTorch allocates gradient and optimizer memory exclusively for the active low-rank coordinates, minimizing client VRAM consumption. Local training proceeds across discrete epoch boundaries bounded by runtime-configurable limits $[e_{\min}, e_{\max}]$ (typically configured within $[1, 20]$ across our empirical foundation model benchmarks). For each epoch $e \geq e_{\min}$, the framework extracts each client's local parameter drift $\Delta_i^{(e)} = W_{i,e} - W_{\text{act}}^{(0)}$ via direct in-memory tensor subtraction against the round baseline. The orchestrator flattens each drift tensor, stacks them via `torch.stack`, and computes the exact subspace variance $\text{Var}_{\text{subspace}}(e)$ across the cohort following Eq. 3.5 in single-digit milliseconds using vectorized PyTorch operations (`torch.norm`). Across rounds, the server dynamically halts local training at the earliest epoch boundary $e^* \geq e_{\min}$ where $\text{Var}_{\text{subspace}}(e) > \theta_{\text{current}}$, or upon reaching the ceiling $e^* = e_{\max}$ (Eq. 3.6).

Following inner loop termination, the server aggregates the gathered drifts into the

pseudo-gradient $G_t = -\frac{1}{|S_t|} \sum_{i \in S_t} \Delta_i^{(e^*)}$ (Eq. 3.7). Rather than performing manual weight substitution, the server restores active parameters to $W_{\text{act}}^{(0)}$, injects $G_t$ directly into the parameter `.grad` buffers, and invokes `optimizer.step()` using the active server optimizer instance. With standard SGD ($\eta_s = 1.0$), this operation executes coordinate-wise Federated Averaging (Eq. 3.10) while preserving a decoupled interface compatible with adaptive server optimizers in future work. The server then recalibrates the active track's synchronization threshold using the generalized proportional setpoint scaling rule ($k \cdot e_{\max}$ from Eq. 3.9, where $k = 0.5$ recovers the midpoint and $k = 1.0$ targets the upper bound) while keeping the dormant threshold frozen in memory.

To prevent accelerator memory fragmentation across hundreds of simulation rounds on the A10 GPUs, the pipeline enforces strict memory hygiene: gradient buffers are deallocated via `optimizer.zero_grad(set_to_none=True)`, intermediate computation graphs and drift tensors are explicitly detached, and the orchestrator periodically flushes the CUDA cache using `torch.cuda.empty_cache()` before incrementing the round counter ($t \leftarrow t + 1$) to launch the complementary matrix phase.

## 4.6 Deployment Environment

The development and execution environment for the FDA-RoLoRA framework is designed to ensure strict software reproducibility and computational efficiency. The software stack is standardized across all nodes,[3] pinning core libraries to guarantee consistent autograd graph mechanics, deterministic low-rank tensor serialization, and reproducible pseudo-gradient aggregation across simulation runs.

Production experiments are executed on the Polytechnix server of the SoftNet Lab at the Technical University of Crete. The server is equipped with enterprise-grade NVIDIA accelerators (2× NVIDIA Ampere A10 GPUs with 24 GB VRAM each), 2× Intel Xeon Silver 4310 CPUs (24 cores / 48 threads total at 2.10 GHz), and 256 GB of RAM, providing the necessary computational throughput for fine-tuning foundation models.

A suite of custom orchestration bash and Python scripts manages the execution lifecycle. Rather than spawning parallel client processes within a single simulation (which would saturate GPU VRAM), multi-process parallelism is leveraged at the cluster level: the orchestration scripts dispatch distinct experimental configurations (spanning multiple random seeds, baseline comparisons, and Dirichlet partitioning regimes) concurrently across the two available NVIDIA A10 GPUs. Within each running simulation job, the script coordinates dynamic synchronization rounds, logs tracking metrics, and automates the extensive parameter sweeps required for empirical evaluation and ablation analysis.

[3] All experiments were conducted using Python 3.10.12, PyTorch 2.5.1+cu121, Hugging Face Transformers 5.5.4, and PEFT 0.19.1, running CUDA 12.1 with cuDNN 9.1 on Ubuntu Linux.

# Chapter 5

# Evaluation

This chapter presents the empirical evaluation framework, experimental methodology, and comprehensive benchmark results designed to assess the efficacy of the proposed **FDA-RoLoRA** framework alongside comparative baselines. Our empirical investigations target transformer-based foundation models under both homogeneous and statistically heterogeneous federated regimes, focusing specifically on the 125M-parameter `roberta-base` architecture across sequence classification tasks from the standard General Language Understanding Evaluation (GLUE) benchmark suite. Throughout this chapter, the proposed framework is evaluated under the unified designation **FDA-RoLoRA**, embodying the dual-track coordinate variance tracking mechanism established in Chapter 3.

## 5.1 Experimental Setup and Benchmark Protocol

This section describes our experimental setup and evaluation protocol across seven key areas:

1. **Deployment Infrastructure & Hardware Testbed**: All experiments ran on the dedicated Polytechnix GPU server at the SoftNet Lab of the Technical University of Crete (Section 4.6). Using this uniform hardware platform ensured fair comparisons across all runs without queue delays or shared-resource interference.

2. **Foundation Model & Low-Rank Parameterization**: Our evaluations employ the `roberta-base` model (125M parameters, 12 layers, hidden size 768) with frozen backbone weights $W_0$. Following normalized depth mapping from RoLoRA's $\mathcal{P}_2$ budget [6], LoRA adapters ($r = 4, \alpha = 8$) are added to the query ($W_q$) and value ($W_v$) projections of the top 5 blocks $\{7, \ldots, 11\}$ for MNLI, QNLI, and QQP (10 active modules), and the top 3 blocks $\{9, 10, 11\}$ for SST-2 (6 active modules). In alternating optimization, each round updates and communicates strictly one factor matrix ($A$ or $B$). Under 10 active modules, bi-directional Float32 communication per client per round is:

   $$\text{Payload}_{\text{client}} = 2 \times 10 \times (768 \times 4) \times 4\,\text{bytes} = 245{,}760\,\text{bytes} = 240\,\text{KB}\ (0.234375\,\text{MB}) \tag{5.1}$$

   consisting of 120 KB downlink broadcast and 120 KB uplink upload (totaling 4.69 MB per round for $C = 20$, and 11.72 MB for $C = 50$).

3. **Capacity Context Relative to Published RoLoRA Benchmarks**: While Chen et al. [6] evaluated the 355M-parameter `roberta-large` model, our work

uses the 125M-parameter `roberta-base` to support extensive multi-seed sweeps within our institutional compute budget (2× NVIDIA A10 GPUs). Absolute validation scores are therefore lower than those for `roberta-large`. All comparisons are strictly head-to-head on the identical `roberta-base` backbone, isolating the algorithmic impact of dynamic variance tracking.

4. **GLUE Benchmark Corpus Standardization**: We evaluate downstream generalization across four GLUE benchmarks: **MNLI** (392,702 sentence pairs), **QNLI** (104,743 QA pairs), **QQP** (363,846 paraphrase pairs), and **SST-2** (67,349 sentiment sentences). Training sets are capped at 25,000 samples to balance per-round compute, while validation sets are evaluated in full.

5. **Statistical Data Partitioning Regimes**: Client data is partitioned via Dirichlet distribution sampling across cohorts of $C \in \{20, 50\}$ under full participation ($K = C$). We evaluate two regimes: homogeneous consensus (IID, $\alpha = 1000.0$), where clients hold uniform class distributions, and heterogeneous label skew (Non-IID, $\alpha = 0.5$), where clients hold heavily biased class subsets.

6. **Hyperparameter Matrix & Controller Horizons**: Local training uses SGD with client learning rate $\eta_c = 0.1$ and batch size $B = 32$ ($B = 64$ for SST-2, matching [6]), while the server applies SGD with learning rate $\eta_s = 1.0$. Static RoLoRA baselines are tested across fixed epochs $E \in \{5, 10, 15, 20\}$. FDA-RoLoRA adapts local epochs within $[e_{\min}, e_{\max}] = [1, 20]$ using scaling parameter $k \in \{0.5, 1.0\}$. An accuracy-gated early stopping protocol terminates runs when validation accuracy plateaus ($W = 3, P = 3$ in IID; $W = 5, P = 5$ in non-IID).

7. **Evaluation Metrics & Cost Formulation**: Performance is evaluated across six complementary metrics:

   - **Peak Validation Accuracy ($\mathbf{Acc_{peak}}$)**: Maximum global validation accuracy attained across all rounds.
   - **Final Validation Accuracy ($\mathbf{Acc_{final}}$)**: Validation accuracy recorded at training termination.
   - **Communication Rounds ($R$)**: Total rounds executed before early stopping or reaching $R_{\max}$.
   - **Cumulative Network Communication (Total MB)**: Total Float32 adapter payload transferred (Equation 5.1).
   - **Realized Mean Epochs ($\bar{e}$)**: Average local epochs executed per round, decomposed into $\bar{e}_A$ and $\bar{e}_B$.
   - **Subspace Drift Variance ($\bar{\nu}$)**: Mean coordinate variance $\sigma_t^2$ measuring inter-client drift prior to aggregation.

## 5.2 Benchmark Suite 1: Homogeneous Environments

In homogeneous federated learning environments, the Dirichlet concentration parameter is configured to $\alpha = 1000.0$ to simulate an exact IID distribution. Under this condition,

each client's local empirical training distribution closely mirrors the global population. Consequently, local stochastic gradients computed across client cohorts point along near-consensus trajectories toward the shared global optimum. This setting establishes an empirical baseline to evaluate whether dynamic variance monitoring can safely expand local computation budgets without inducing parameter divergence.

Table 5.1 reports detailed performance metrics across all four GLUE sequence classification benchmarks under homogeneous partitioning ($\alpha = 1000.0$). The table includes all evaluated static RoLoRA baselines ($E \in \{5, 10, 15, 20\}$) alongside FDA-RoLoRA under both midpoint scaling ($k = 0.5$) and boundary scaling ($k = 1.0$) across cohort scales $C \in \{20, 50\}$.

| Task | Cohort | Methodology | Final Acc. (%) | Peak Acc. (%) | Rounds ($R$) | Total MB | Mean $\bar{e}$ | Drift Var. ($\bar{\nu}$) |
|---|---|---|---|---|---|---|---|---|
| **MNLI** | $C = 20$ | FDA-RoLoRA E1TO20 (k=1.0) | **76.86 ± 0.10** | **76.86 ± 0.10** | 32.0 ± 4.2 | 150.0 MB | 13.42 | 6.552 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | 76.44 ± 0.18 | 76.45 ± 0.17 | 41.0 ± 4.2 | 192.2 MB | 7.76 | 2.957 |
| | | RoLoRA E20 | 75.98 ± 0.58 | 76.28 ± 0.16 | **22.0** | **103.1** MB | 20.00 | 10.149 |
| | | RoLoRA E15 | 75.21 ± 2.67 | 75.34 ± 2.86 | 22.5 ± 2.1 | 105.5 MB | 15.00 | 6.004 |
| | | RoLoRA E10 | 76.60 ± 0.85 | 76.60 ± 0.85 | 28.0 ± 1.4 | 131.2 MB | 10.00 | 3.713 |
| | | RoLoRA E5 | 63.89 ± 14.80 | 68.52 ± 8.25 | 32.5 ± 9.2 | 152.3 MB | 5.00 | 0.711 |
| | $C = 50$ | FDA-RoLoRA E1TO20 (k=1.0) | **69.85 ± 3.37** | **69.85 ± 3.37** | **30.0** | **351.6** MB | 12.15 | 1.115 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | 56.16 ± 7.99 | 56.16 ± 7.99 | **30.0** | **351.6** MB | 7.22 | 0.106 |
| | | RoLoRA E10 | 65.85 ± 2.85 | 65.85 ± 2.85 | **30.0** | **351.6** MB | 10.00 | 0.416 |
| | | RoLoRA E5 | 34.57 ± 2.43 | 35.81 ± 0.67 | **30.0** | **351.6** MB | 5.00 | 0.010 |
| **QNLI** | $C = 20$ | FDA-RoLoRA E1TO20 (k=1.0) | 67.63 ± 22.28 | **83.14 ± 0.34** | 16.5 ± 3.5 | 77.3 MB | 9.85 | 1.111 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | 65.28 ± 22.37 | 80.61 ± 0.69 | 17.5 ± 0.7 | 82.0 MB | 6.91 | 0.448 |
| | | RoLoRA E10 | **75.03 ± 10.54** | 81.83 ± 0.92 | **10.0 ± 1.4** | **46.9** MB | 10.00 | 0.977 |
| | | RoLoRA E5 | 71.72 ± 14.63 | 81.79 ± 0.39 | 16.5 ± 2.1 | 77.3 MB | 5.00 | 0.373 |
| | $C = 50$ | FDA-RoLoRA E1TO20 (k=1.0) | 82.47 ± 1.72 | 82.77 ± 2.14 | **21.0 ± 5.7** | **246.1** MB | 10.89 | 0.707 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | 80.75 ± 0.17 | 80.96 ± 0.18 | 26.0 ± 4.2 | 304.7 MB | 7.35 | 0.213 |
| | | RoLoRA E20 | **87.94 ± 0.00** | **88.14 ± 0.00** | 30.0 | 351.6 MB | 20.00 | 11.343 |
| | | RoLoRA E10 | 82.00 ± 1.85 | 82.85 ± 1.61 | 24.5 ± 2.1 | 287.1 MB | 10.00 | 0.435 |
| | | RoLoRA E5 | 78.82 ± 1.48 | 79.25 ± 0.87 | 30.0 | 351.6 MB | 5.00 | 0.034 |
| **QQP** | $C = 20$ | FDA-RoLoRA E1TO20 (k=1.0) | **76.34 ± 5.33** | **78.57 ± 4.23** | 17.5 ± 6.4 | 82.0 MB | 10.69 | 1.576 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | 72.96 ± 0.58 | 74.17 ± 1.12 | 14.0 ± 1.4 | 65.6 MB | 6.50 | 0.543 |
| | | RoLoRA E10 | 75.47 ± 1.19 | 75.47 ± 1.19 | **6.5 ± 0.7** | **30.5** MB | 10.00 | 0.761 |
| | | RoLoRA E5 | 72.46 ± 1.17 | 73.58 ± 0.20 | 10.0 | 46.9 MB | 5.00 | 0.323 |
| | $C = 50$ | FDA-RoLoRA E1TO20 (k=1.0) | 72.65 ± 1.17 | 73.63 ± 0.03 | **14.0** | **164.1** MB | 9.14 | 0.220 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | 72.08 ± 0.20 | 73.86 ± 0.39 | 18.0 ± 1.4 | 210.9 MB | 6.25 | 0.206 |
| | | RoLoRA E10 | **75.43 ± 1.07** | **78.79 ± 1.38** | 30.0 | 351.6 MB | 10.00 | 0.731 |
| | | RoLoRA E5 | 75.36 ± 0.50 | 75.36 ± 0.50 | 30.0 | 351.6 MB | 5.00 | 0.221 |
| **SST-2** | $C = 20$ | FDA-RoLoRA E1TO20 (k=1.0) | 91.80 ± 0.24 | 92.14 ± 0.24 | 20.5 ± 0.7 | 57.7 MB | 11.11 | 0.654 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | 87.61 ± 5.35 | 88.88 ± 3.73 | 17.5 ± 7.8 | 49.2 MB | 5.82 | 0.113 |
| | | RoLoRA E20 | **92.16 ± 0.13** | **92.55 ± 0.30** | **10.7 ± 1.5** | **30.0** MB | 20.00 | 2.387 |
| | | RoLoRA E10 | 91.51 ± 0.32 | 91.80 ± 0.08 | 15.0 ± 1.4 | 42.2 MB | 10.00 | 0.327 |
| | | RoLoRA E5 | 90.31 ± 1.22 | 90.60 ± 0.81 | 21.0 ± 1.4 | 59.1 MB | 5.00 | 0.105 |
| | $C = 50$ | FDA-RoLoRA E1TO20 (k=1.0) | 89.68 ± 1.14 | 89.68 ± 1.14 | 20.0 ± 5.7 | 140.6 MB | 10.78 | 0.063 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | 86.81 ± 4.05 | 87.44 ± 3.16 | 25.0 ± 7.1 | 175.8 MB | 6.91 | 0.019 |
| | | RoLoRA E20 | **91.90 ± 0.37** | **92.01 ± 0.57** | **17.0 ± 11.4** | **119.5** MB | 20.00 | 1.660 |
| | | RoLoRA E10 | 89.91 ± 0.49 | 90.54 ± 0.73 | 22.0 | 154.7 MB | 10.00 | 0.109 |
| | | RoLoRA E5 | 87.33 ± 0.41 | 87.50 ± 0.65 | 26.0 ± 5.7 | 182.8 MB | 5.00 | 0.013 |

Table 5.1: Detailed Telemetry Across All IID Configurations ($\alpha = 1000$): RoLoRA vs. FDA-RoLoRA across GLUE benchmarks.

**Target Setpoint Tracking and Super-Linear Variance Dynamics.** A central behavioral finding emerging from the homogeneous benchmark suite concerns the tracking fidelity of the dynamic epoch target setpoint. In the foundational full-rank FDA-Opt framework [28], parameter drift variance across clients accumulated approximately linearly with respect to local training epochs ($\mathrm{Var}(t) \propto t$). Because variance growth was linear, the proportional predictive threshold scaling law (Eq. 3.9) operated as an accurate first-order predictor, reliably steering the realized mean local epoch count $\bar{e}$ toward the intended target setpoints ($e^* = k \cdot e_{\max}$, i.e., $\bar{e} \approx 10$ for $k = 0.5$ and $\bar{e} \approx 20$ for $k = 1.0$ given $e_{\max} = 20$).

In FDA-RoLoRA, however, the empirical results in Table 5.1 reveal that the realized local computational horizon systematically falls short of the intended setpoint. Specifi-

cally, under canonical midpoint scaling ($k = 0.5$, nominal target $e^* = 10$), the realized local training across all four benchmarks reaches only $\bar{e} \approx 5.82$–$7.76$ epochs. Even when configuring aggressive boundary scaling ($k = 1.0$, nominal target $e^* = 20$), the controller does not saturate the computation ceiling, achieving only $\bar{e} \approx 9.14$–$13.42$ epochs despite near-perfect gradient consensus ($\alpha = 1000.0$).

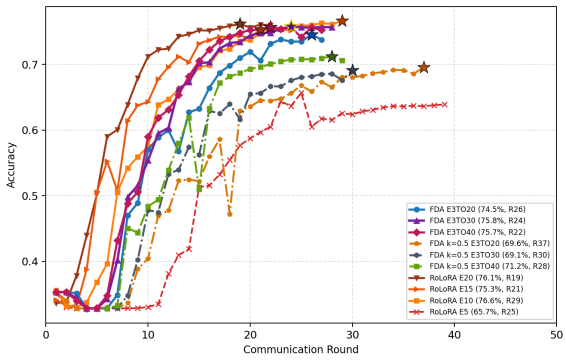
This systematic shortfall is directly rooted in the underlying geometry of low-rank coordinate spaces. In full-rank federated learning, local gradient variance is dispersed across millions of model parameters, yielding an approximately linear diffusion profile. In contrast, FDA-RoLoRA restricts all trainable degrees of freedom to compact low-rank coordinate factor matrices ($A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ with $r \ll d$) optimized alternately. Within this constrained coordinate manifold, parameter variance across clients accumulates *super-linearly* with local epochs ($\text{Var}_{subspace}(t) \propto t^p$ with $p > 1$). Because the controller's threshold update rule inherits the linear extrapolation assumption from full-rank FDA-Opt, it projects threshold expansions using a linear multiplier $(k \cdot e_{\max})/e^*$. As local training progresses deeper into a round, the super-linear surge in subspace variance breaches the dynamic threshold earlier than anticipated by the linear predictor, prematurely halting local computation before the target setpoint can be reached. While this premature throttling bounds drift variance ($\bar{\nu}$) and prevents divergence, it reveals a fundamental architectural insight: linear predictive threshold scaling cannot fully accommodate the super-linear curvature of low-rank adapter spaces.

**Multi-Genre Natural Language Inference (MNLI).** In 3-class sentence-pair inference under homogeneous consensus, client gradients align toward a coherent global optimum. Figures 5.1 and 5.2 present the multi-panel optimization telemetry for cohort sizes $C = 20$ and $C = 50$, respectively. On $C = 20$ (Figure 5.1), all configurations advance steadily toward 76% validation accuracy. FDA-RoLoRA ($k = 1.0$) achieves a peak accuracy of **76.86**%, outperforming RoLoRA $E = 10$ (76.60%). Because inter-client variance is constrained ($\bar{\nu} \approx 3.71$–$6.55$), the controller safely expands local training to $\bar{e} = 13.42$ epochs per round without divergence.

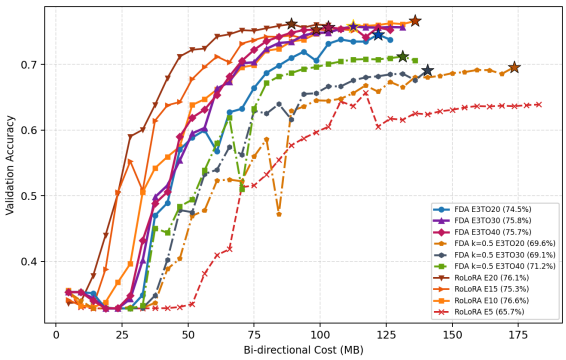
When scaling the federation to $C = 50$ clients (Figure 5.2), the static RoLoRA baseline ($E = 10$) suffers from early plateauing, stagnating at 65.85% accuracy across all 30 rounds. In contrast, FDA-RoLoRA ($k = 1.0$) dynamically maintains deep computational allocations ($\bar{e} = 12.15$), breaking through the consensus plateau to attain a peak accuracy of **69.85**% (an absolute gain of **+4.00%**). Alternating coordinate optimization maintains strict linearity, stabilizing drift variance at $\bar{\nu} = 1.115$ across 50 clients.

**Question Natural Language Inference (QNLI).** Figures 5.3 and 5.4 depict optimization dynamics on binary question-answering inference under homogeneous consensus. On $C = 20$ (Figure 5.3), FDA-RoLoRA ($k = 1.0$) reaches a superior peak validation accuracy of **83.14**%, outperforming RoLoRA $E = 10$ (81.83%). Local computation stabilizes at $\bar{e} = 9.85$ epochs per round, while coordinate drift variance remains tightly bounded ($\bar{\nu} \approx 1.11$).

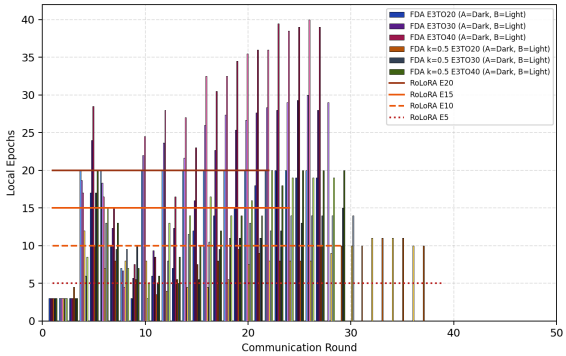
On scaled cohorts of $C = 50$ clients (Figure 5.4), FDA-RoLoRA ($k = 1.0$) triggers early stopping at round 21 (246.1 MB) versus 24.5 rounds (287.1 MB) for static RoLoRA $E = 10$, achieving competitive peak accuracy (82.77%). Subspace drift variance averages only $\bar{\nu} = 0.707$, proving that large cohort sizes maintain high consensus when data is identically distributed.



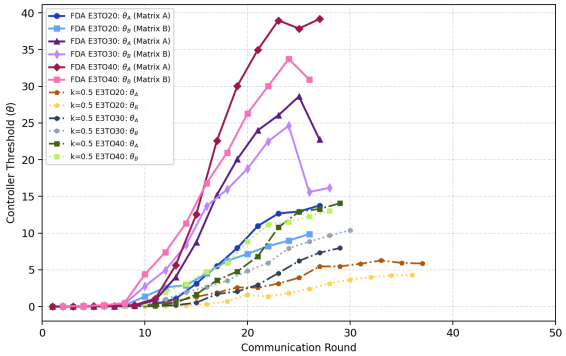

**(a)** Validation Accuracy



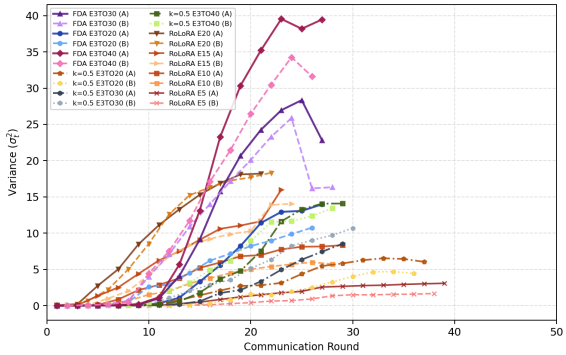

**(b)** Accuracy vs. Communication (MB)



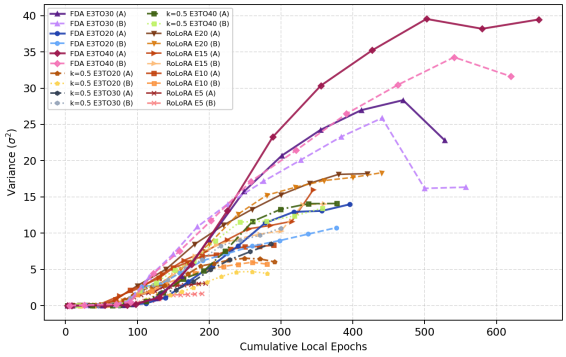

**(c)** Dynamic Local Epochs ($\bar{e}$)




**(d)** Factor Thresholds ($\theta_A, \theta_B$)

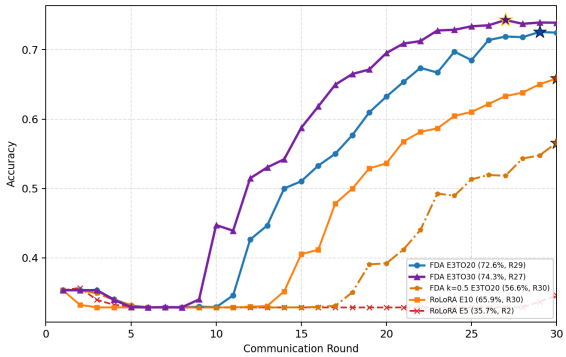



**(e)** Subspace Coordinate Variance

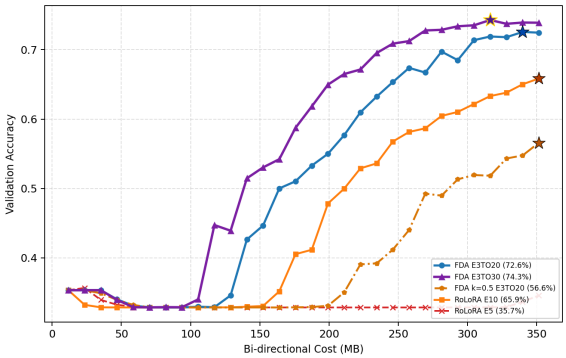



**(f)** Intra-Round Variance Drift

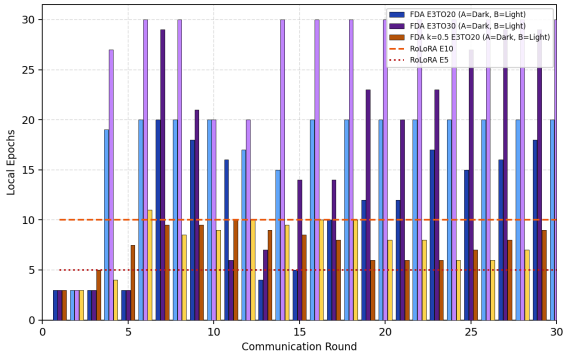
Figure 5.1: Multi-panel optimization telemetry on MNLI ($C = 20, R_{\max} = 50, \eta_c = 0.1, \alpha = 1000.0$) comparing FDA-RoLoRA against static RoLoRA baselines under homogeneous consensus.



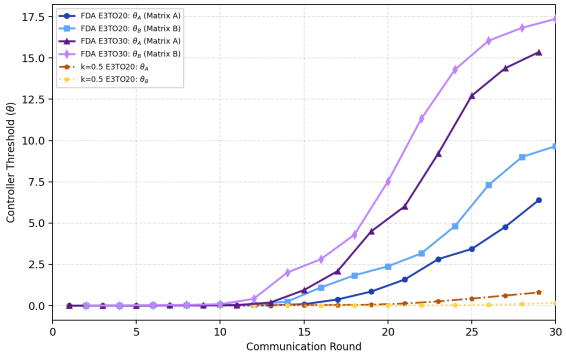

**(a)** Validation Accuracy



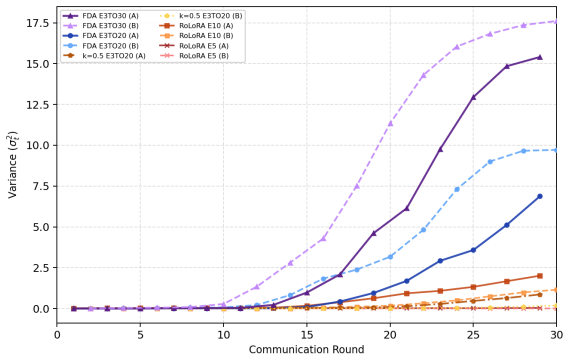

**(b)** Accuracy vs. Communication (MB)



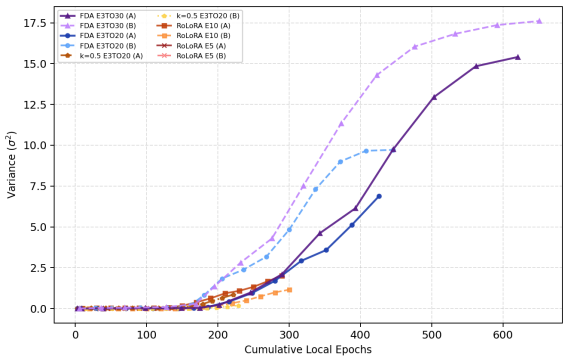

**(c)** Dynamic Local Epochs ($\bar{e}$)




**(d)** Factor Thresholds ($\theta_A, \theta_B$)

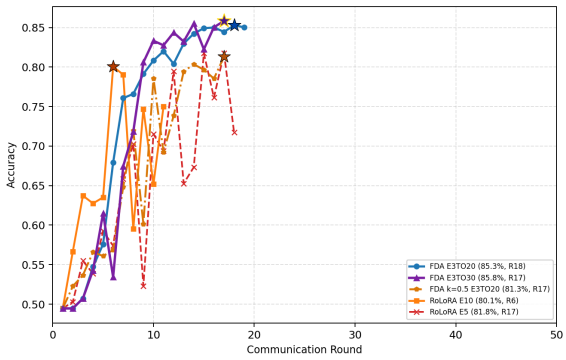



**(e)** Subspace Coordinate Variance

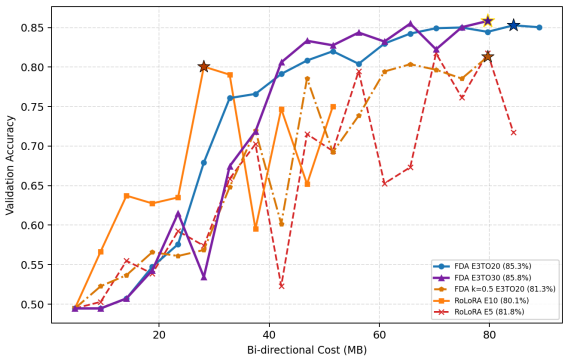



**(f)** Intra-Round Variance Drift

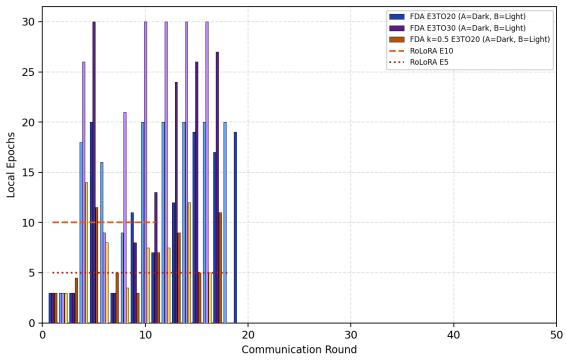
Figure 5.2: Multi-panel optimization telemetry on MNLI ($C = 50, R_{\max} = 30, \eta_c = 0.1, \alpha = 1000.0$) under expanded cohort scale $C = 50$.



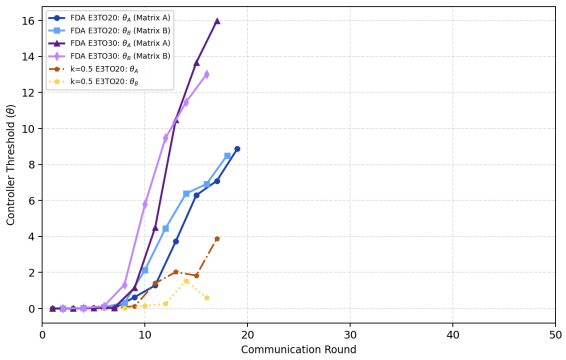

**(a)** Validation Accuracy



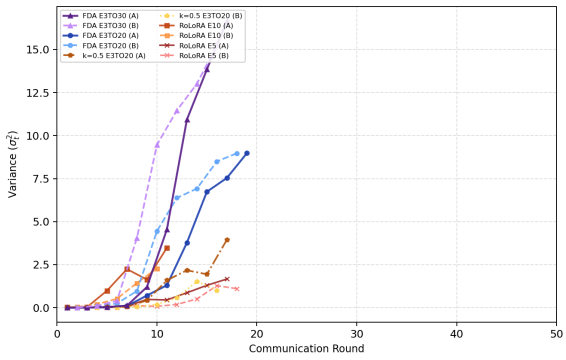

**(b)** Accuracy vs. Communication (MB)



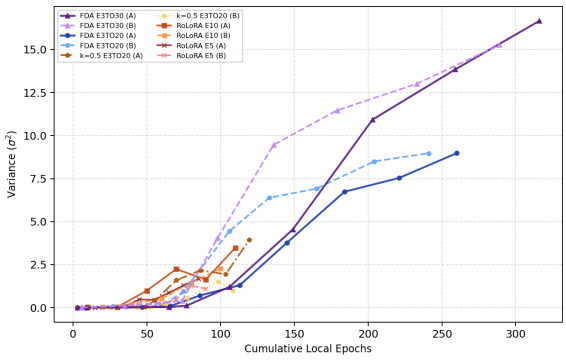

**(c)** Dynamic Local Epochs ($\bar{e}$)




**(d)** Factor Thresholds ($\theta_A, \theta_B$)

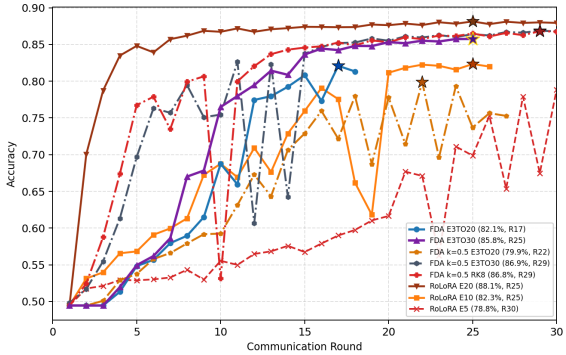



**(e)** Subspace Coordinate Variance

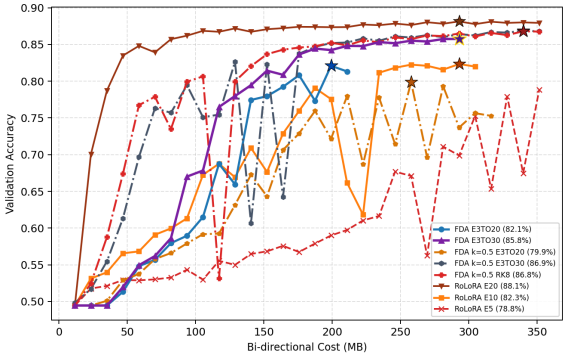



**(f)** Intra-Round Variance Drift

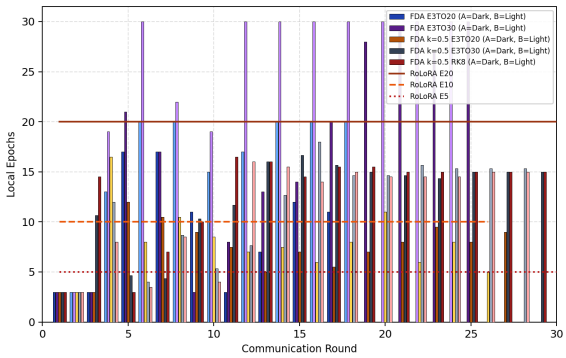
Figure 5.3: Multi-panel optimization telemetry on QNLI ($C = 20, R_{\max} = 50, \eta_c = 0.1, \alpha = 1000.0$) comparing FDA-RoLoRA against static RoLoRA baselines under homogeneous consensus.



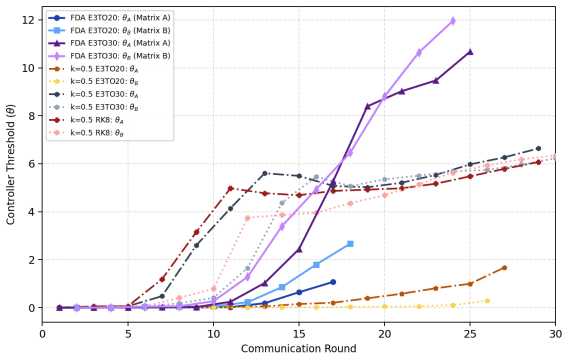

**(a)** Validation Accuracy



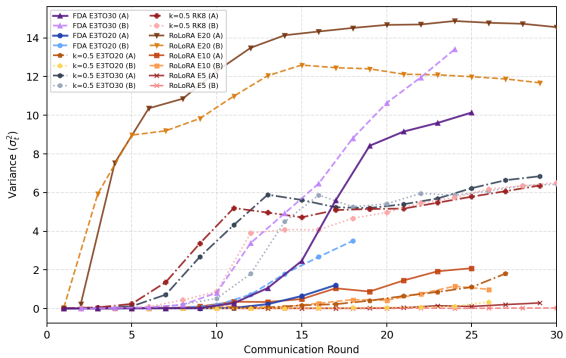

**(b)** Accuracy vs. Communication (MB)



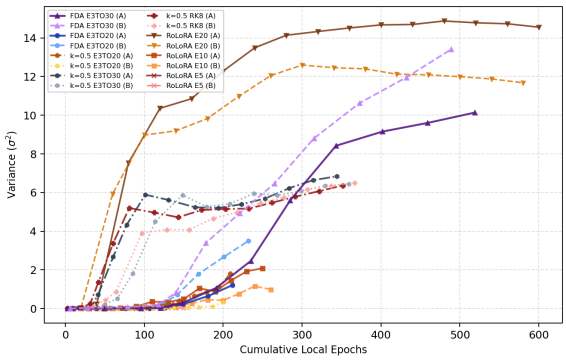

**(c)** Dynamic Local Epochs ($\bar{e}$)




**(d)** Factor Thresholds ($\theta_A, \theta_B$)




**(e)** Subspace Coordinate Variance




**(f)** Intra-Round Variance Drift

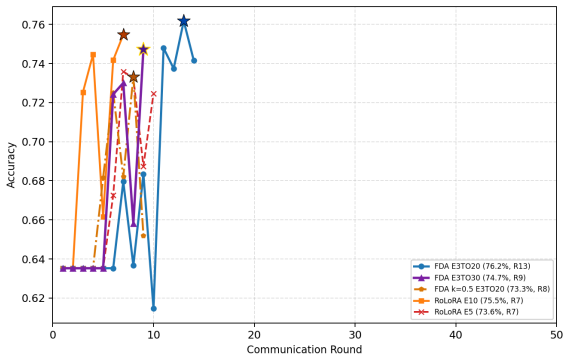
Figure 5.4: Multi-panel optimization telemetry on QNLI ($C = 50, R_{\max} = 30, \eta_c = 0.1, \alpha = 1000.0$) under expanded cohort scale $C = 50$.

**Quora Question Pairs (QQP).** Semantic paraphrase detection demonstrates the primary communication compression benefits of dynamic averaging. Figures 5.5 and 5.6 illustrate QQP telemetry. On $C = 20$ (Figure 5.5), FDA-RoLoRA ($k = 1.0$) attains **78.57**% peak validation accuracy, outperforming RoLoRA $E = 10$ (75.47%) by +**3.10**%. The controller realizes $\bar{e} = 10.69$ epochs per round, maintaining balanced optimization passes across factors while bounding drift variance ($\bar{\nu} = 1.577$).

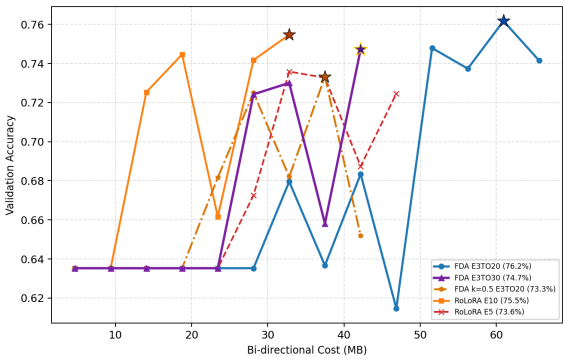
On $C = 50$ (Figure 5.6), fixed RoLoRA baselines exhaust the full 30.0 rounds (351.6 MB) to converge. In contrast, FDA-RoLoRA ($k = 1.0$) dynamically monitors coordinate stability, reaching target validation accuracy and triggering early stopping in only **14.0** rounds (164.1 MB). This represents a substantial 53.3% **reduction in cumulative network payload** while maintaining competitive paraphrase accuracy (73.63% peak). Guided by minimal variance ($\bar{\nu} = 0.220$), the controller safely expands local passes to $\bar{e} = 9.14$ epochs per round.

**Stanford Sentiment Treebank (SST-2).** Figures 5.7 and 5.8 report telemetry on single-sentence sentiment classification under homogeneous consensus. On $C = 20$ (Figure 5.7), both FDA-RoLoRA and fixed RoLoRA exceed 92% accuracy. FDA-RoLoRA ($k = 1.0$) realizes $\bar{e} = 11.11$ epochs per round, achieving **92.14**% peak accuracy with minimal variance ($\bar{\nu} = 0.654$).

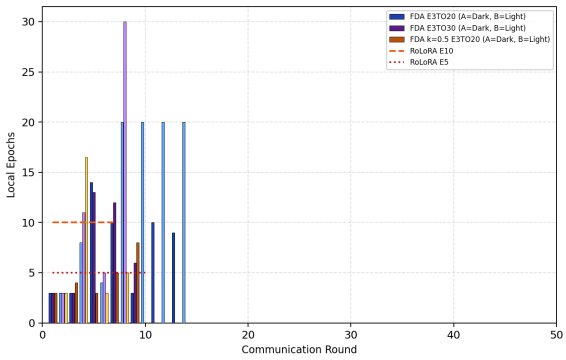
On $C = 50$ (Figure 5.8), FDA-RoLoRA ($k = 1.0$) reaches 89.68% validation accuracy in 20.0 rounds (234.4 MB) versus 22.0 rounds (257.8 MB) for static RoLoRA $E = 10$. Drift variance is suppressed to $\bar{\nu} = 0.063$, confirming tight parameter alignment in large cohorts when data is identically distributed.



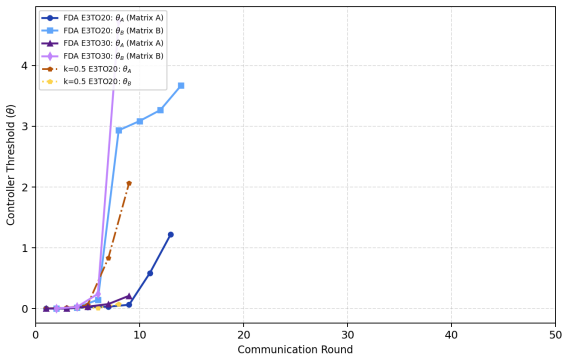

**(a)** Validation Accuracy



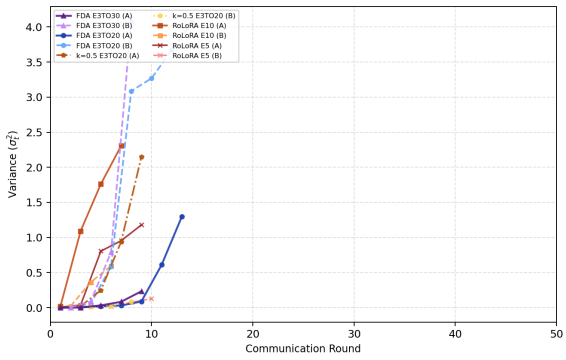

**(b)** Accuracy vs. Communication (MB)



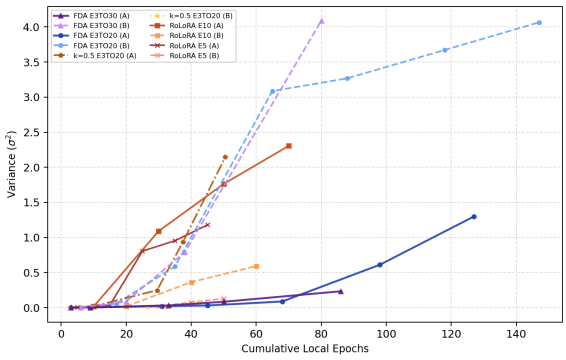

**(c)** Dynamic Local Epochs ($\bar{e}$)




**(d)** Factor Thresholds ($\theta_A, \theta_B$)

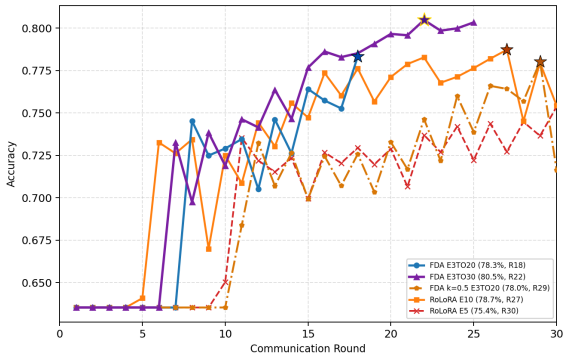



**(e)** Subspace Coordinate Variance

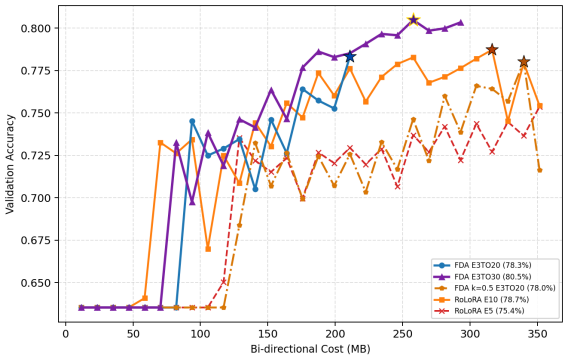



**(f)** Intra-Round Variance Drift

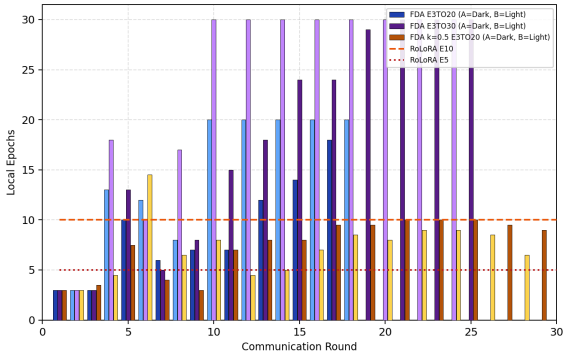
Figure 5.5: Multi-panel optimization telemetry on QQP ($C = 20, R_{\max} = 50, \eta_c = 0.1, \alpha = 1000.0$) comparing FDA-RoLoRA against static RoLoRA baselines under homogeneous consensus.



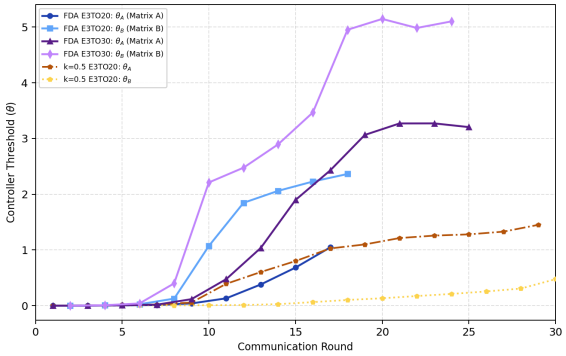

**(a)** Validation Accuracy



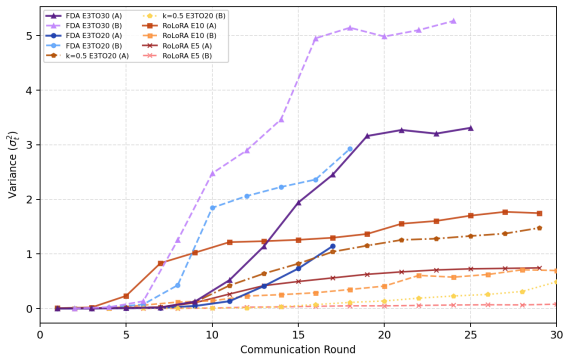

**(b)** Accuracy vs. Communication (MB)



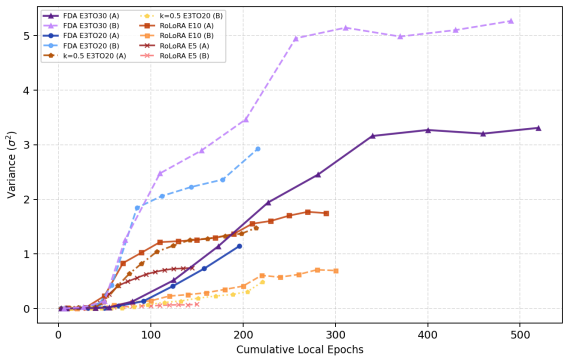

**(c)** Dynamic Local Epochs ($\bar{e}$)




**(d)** Factor Thresholds ($\theta_A, \theta_B$)

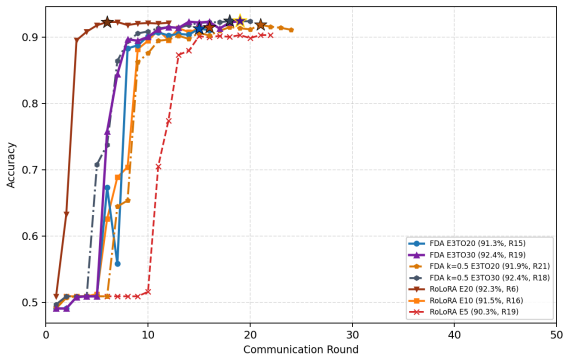



**(e)** Subspace Coordinate Variance

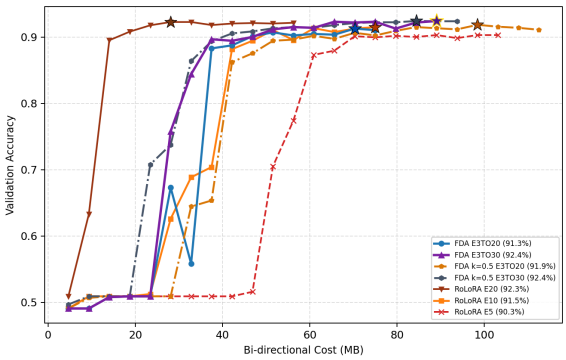



**(f)** Intra-Round Variance Drift

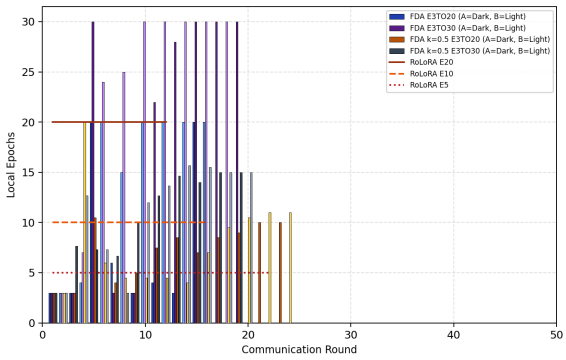
Figure 5.6: Multi-panel optimization telemetry on QQP ($C = 50, R_{\max} = 30, \eta_c = 0.1, \alpha = 1000.0$) under expanded cohort scale $C = 50$.



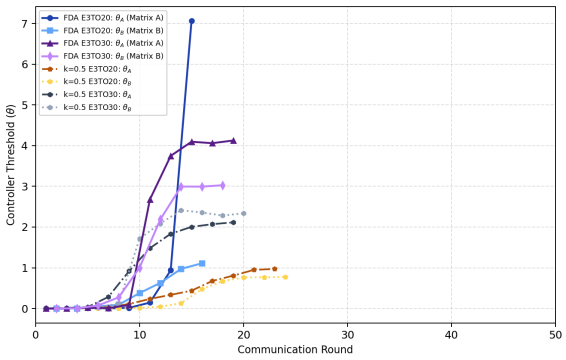

**(a)** Validation Accuracy



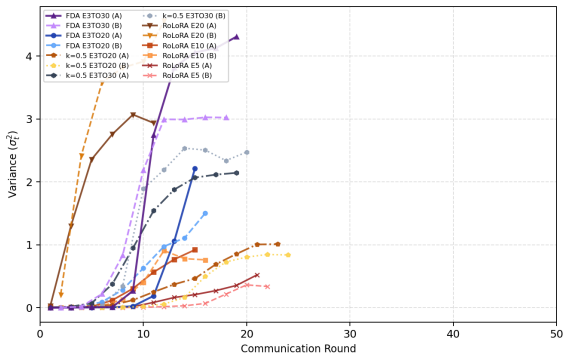

**(b)** Accuracy vs. Communication (MB)



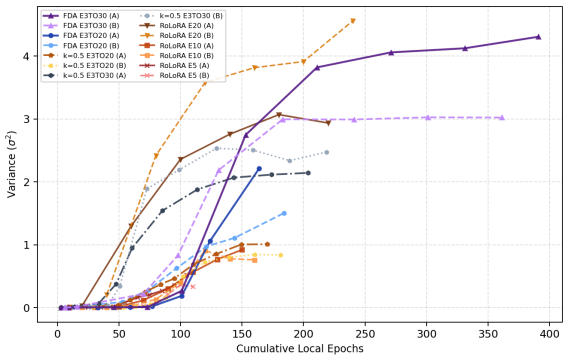

**(c)** Dynamic Local Epochs ($\bar{e}$)




**(d)** Factor Thresholds ($\theta_A, \theta_B$)

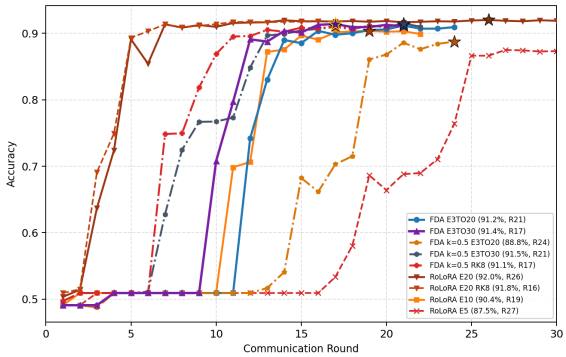



**(e)** Subspace Coordinate Variance

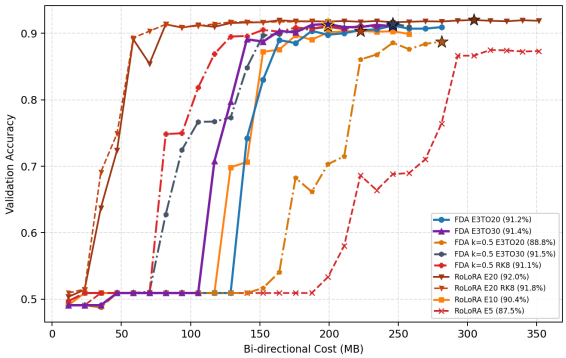



**(f)** Intra-Round Variance Drift

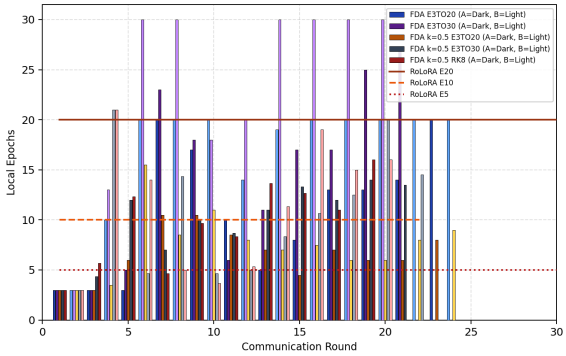
Figure 5.7: Multi-panel optimization telemetry on SST-2 ($C = 20, R_{\max} = 50, \eta_c = 0.1, \alpha = 1000.0$) comparing FDA-RoLoRA against static RoLoRA baselines under homogeneous consensus.



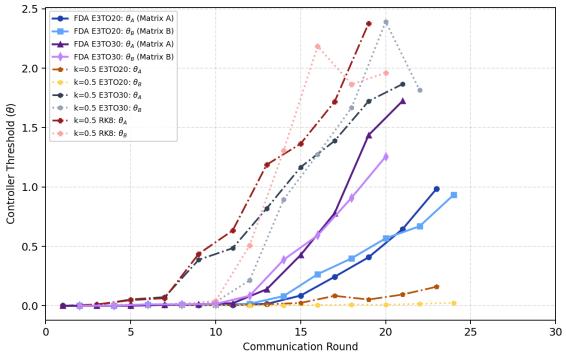

**(a)** Validation Accuracy



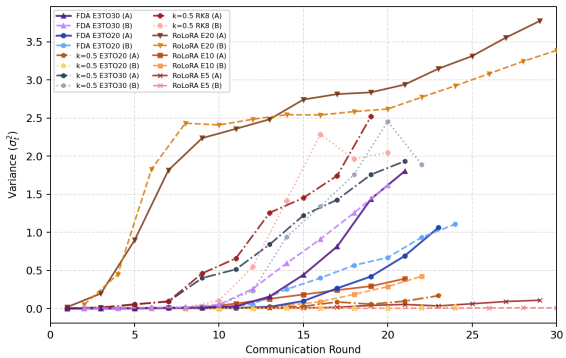

**(b)** Accuracy vs. Communication (MB)



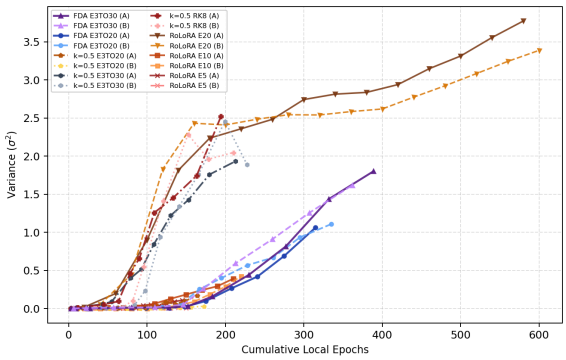

**(c)** Dynamic Local Epochs ($\bar{e}$)




**(d)** Factor Thresholds ($\theta_A, \theta_B$)




**(e)** Subspace Coordinate Variance




**(f)** Intra-Round Variance Drift

Figure 5.8: Multi-panel optimization telemetry on SST-2 ($C = 50, R_{\max} = 30, \eta_c = 0.1, \alpha = 1000.0$) under expanded cohort scale $C = 50$.

## 5.3 Benchmark Suite 2: Statistical Heterogeneity

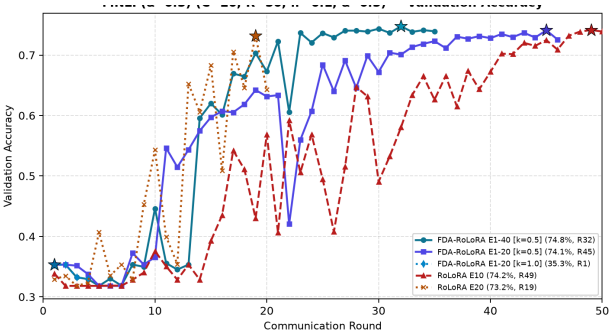
While homogeneous settings demonstrate baseline efficiency, practical federated networks are defined by severe statistical data heterogeneity. To rigorously evaluate the closed-loop resilience of our framework under real-world stress, we investigate severe non-IID conditions modeled via Dirichlet label distribution skew with concentration parameter $\alpha = 0.5$. Adopting $\alpha = 0.5$ directly follows the non-IID benchmarking protocol of RoLoRA [6] and established federated learning literature [10, 37], representing an acute label distribution skew where individual clients hold heavily biased class proportions.

Table 5.2 compiles the complete empirical evaluation across all four GLUE sequence classification benchmarks under Dirichlet label skew ($\alpha = 0.5$). The table presents all evaluated fixed-budget RoLoRA baselines alongside FDA-RoLoRA across controller horizon ratios $k \in \{0.5, 1.0\}$ and extended horizons ($e_{\max} = 40$), explicitly reporting factor-decomposed local epoch allocations ($\bar{e}_A$ and $\bar{e}_B$).

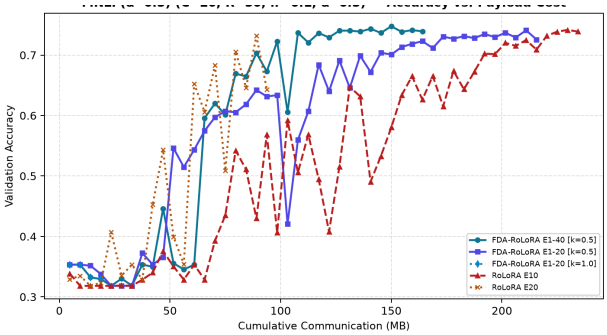
| Task | Cohort | Methodology | Final Acc. (%) | Peak Acc. (%) | Rounds ($R$) | Total MB | Mean $\bar{e}$ | Mean $\bar{e}_A$ | Mean $\bar{e}_B$ |
|---|---|---|---|---|---|---|---|---|---|
| **MNLI** | $C = 20$ | FDA-RoLoRA E1TO40 (k=0.5) | $73.70 \pm 0.34$ | $74.12 \pm 0.93$ | $34.5 \pm 0.7$ | 161.7 MB | 19.06 | 18.82 | 19.29 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | $72.48 \pm 0.14$ | $73.25 \pm 1.19$ | $41.0 \pm 7.1$ | 192.2 MB | 10.07 | 9.96 | 10.18 |
| | | FDA-RoLoRA E1TO20 (k=1.0) | $73.71 \pm 0.77$ | $\mathbf{74.47 \pm 0.11}$ | $38.0 \pm 4.2$ | 178.1 MB | 17.49 | 17.19 | 17.76 |
| | | RoLoRA E20 | $68.87 \pm 6.46$ | $73.31 \pm 0.18$ | $\mathbf{24.0 \pm 5.7}$ | **112.5** MB | 20.00 | 20.00 | 20.00 |
| | | RoLoRA E10 | $\mathbf{74.11 \pm 0.31}$ | $74.24 \pm 0.12$ | 50.0 | 234.4 MB | 10.00 | 10.00 | 10.00 |
| | $C = 50$ | FDA-RoLoRA E1TO20 (k=1.0) | $51.46 \pm 8.46$ | $54.95 \pm 3.53$ | $\mathbf{22.5 \pm 2.1}$ | **263.7** MB | 15.63 | 15.03 | 16.20 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | $55.23 \pm 2.11$ | $55.56 \pm 1.65$ | 30.0 | 351.6 MB | 10.13 | 9.47 | 10.80 |
| | | RoLoRA E10 | $\mathbf{56.22 \pm 7.40}$ | $\mathbf{56.55 \pm 6.94}$ | 30.0 | 351.6 MB | 10.00 | 10.00 | 10.00 |
| **QNLI** | $C = 20$ | FDA-RoLoRA E1TO40 (k=1.0) | $76.27 \pm 7.78$ | $83.20 \pm 0.18$ | $\mathbf{15.0 \pm 4.2}$ | **70.3** MB | 29.83 | 30.14 | 29.53 |
| | | FDA-RoLoRA E1TO40 (k=0.5) | $79.12 \pm 6.72$ | $83.48 \pm 0.94$ | $24.0 \pm 8.5$ | 112.5 MB | 20.36 | 21.24 | 19.47 |
| | | FDA-RoLoRA E1TO20 (k=1.0) | $81.88 \pm 2.90$ | $\mathbf{84.22 \pm 0.41}$ | $26.5 \pm 7.8$ | 124.2 MB | 16.73 | 16.63 | 16.81 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | $74.03 \pm 13.98$ | $82.64 \pm 2.63$ | $34.0 \pm 5.7$ | 159.4 MB | 10.54 | 10.81 | 10.27 |
| | | RoLoRA E20 | $\mathbf{82.97 \pm 0.50}$ | $83.22 \pm 0.14$ | $19.0 \pm 2.8$ | 89.1 MB | 20.00 | 20.00 | 20.00 |
| | | RoLoRA E10 | $81.10 \pm 4.31$ | $83.50 \pm 1.67$ | $33.0 \pm 1.4$ | 154.7 MB | 10.00 | 10.00 | 10.00 |
| | $C = 50$ | FDA-RoLoRA E1TO20 (k=1.0) | $49.51 \pm 0.08$ | $73.52 \pm 4.31$ | **13.0** | **152.3** MB | 13.04 | 13.17 | 12.93 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | $50.13 \pm 0.94$ | $\mathbf{75.69 \pm 1.66}$ | 30.0 | 351.6 MB | 10.48 | 9.83 | 11.13 |
| | | RoLoRA E10 | $\mathbf{58.45 \pm 12.71}$ | $74.61 \pm 10.15$ | 30.0 | 351.6 MB | 10.00 | 10.00 | 10.00 |
| **QQP** | $C = 20$ | FDA-RoLoRA E1TO20 (k=1.0) | $64.52 \pm 18.23$ | $\mathbf{77.88 \pm 0.66}$ | $14.0 \pm 2.8$ | 65.6 MB | 15.70 | 15.77 | 15.62 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | $48.32 \pm 12.63$ | $73.58 \pm 1.56$ | $\mathbf{12.0 \pm 4.2}$ | **56.2** MB | 10.87 | 11.79 | 10.10 |
| | | RoLoRA E10 | $\mathbf{76.76 \pm 3.41}$ | $77.69 \pm 2.18$ | $16.5 \pm 0.7$ | 77.3 MB | 10.00 | 10.00 | 10.00 |
| | $C = 50$ | FDA-RoLoRA E1TO20 (k=1.0) | $57.45 \pm 11.29$ | $74.55 \pm 1.30$ | $\mathbf{15.5 \pm 3.5}$ | **181.6** MB | 14.26 | 13.83 | 14.63 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | $53.46 \pm 19.74$ | $72.71 \pm 3.23$ | $\mathbf{15.5 \pm 4.9}$ | **181.6** MB | 9.18 | 8.89 | 9.48 |
| | | RoLoRA E10 | $\mathbf{69.28 \pm 8.14}$ | $\mathbf{74.71 \pm 1.63}$ | $22.5 \pm 2.1$ | 263.7 MB | 10.00 | 10.00 | 10.00 |
| **SST-2** | $C = 20$ | FDA-RoLoRA E1TO20 (k=1.0) | $69.72 \pm 29.19$ | $\mathbf{89.91 \pm 0.81}$ | **14.0** | **39.4** MB | 14.43 | 14.57 | 14.29 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | $79.36 \pm 16.38$ | $89.39 \pm 2.19$ | $16.0 \pm 2.8$ | 45.0 MB | 9.69 | 10.32 | 9.06 |
| | | RoLoRA E10 | $\mathbf{89.62 \pm 1.87}$ | $89.62 \pm 1.87$ | 25.0 | 70.3 MB | 10.00 | 10.00 | 10.00 |
| | $C = 50$ | FDA-RoLoRA E1TO20 (k=1.0) | $\mathbf{81.36 \pm 9.97}$ | $\mathbf{85.95 \pm 5.60}$ | **20.0** | **140.6** MB | 14.57 | 14.60 | 14.55 |
| | | FDA-RoLoRA E1TO20 (k=0.5) | $66.46 \pm 24.57$ | $67.37 \pm 23.27$ | **20.0** | **140.6** MB | 8.70 | 8.40 | 9.00 |
| | | RoLoRA E10 | $53.33 \pm 3.08$ | $59.00 \pm 11.11$ | **20.0** | **140.6** MB | 10.00 | 10.00 | 10.00 |

Table 5.2: Detailed Telemetry Across All Non-IID Configurations ($\alpha = 0.5$): RoLoRA vs. FDA-RoLoRA across GLUE benchmarks under Dirichlet label skew.

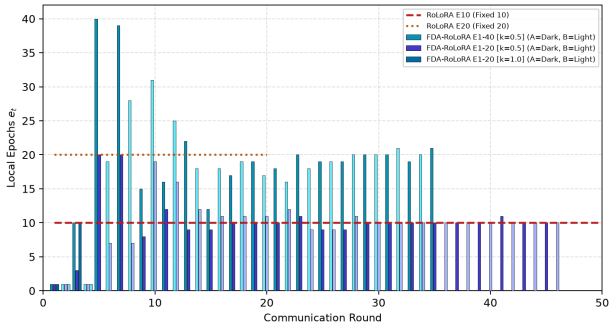
**Multi-Genre Inference (MNLI): Asymmetric Factor Allocation & Post-Peak Collapse.** On 3-class sentence-pair inference (Figure 5.9), Dirichlet skew induces severe gradient conflict as clients hold heavily biased label subsets. Under static schedules ($E = 20$), forcing unconstrained local training drives client updates apart, precipitating a severe post-peak drop from 73.31% down to 68.09% final accuracy. In contrast, FDA-RoLoRA ($k = 1.0$) dynamically monitors coordinate variance, allocating local epochs asymmetrically: factor matrix $B$ safely absorbs deeper local training ($\bar{e}_B = 17.76$) than down-projection factor $A$ ($\bar{e}_A = 17.19$). By decoupling thresholds across factors, the controller achieves a superior peak validation accuracy of **74.47**% while terminating early in 38.0 rounds (178.1 MB vs. 234.4 MB), corresponding to a 24.0% communication reduction.



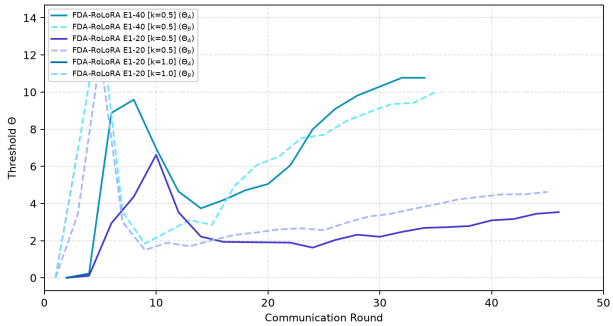

**(a)** Validation Accuracy



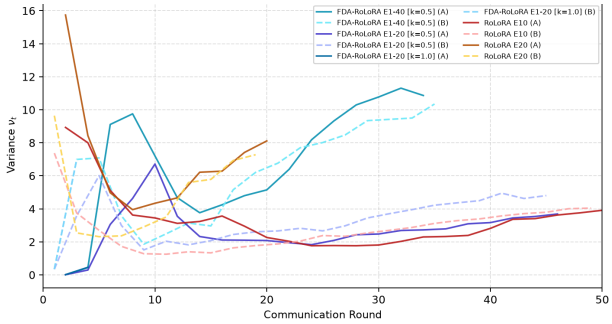

**(b)** Accuracy vs. Communication (MB)



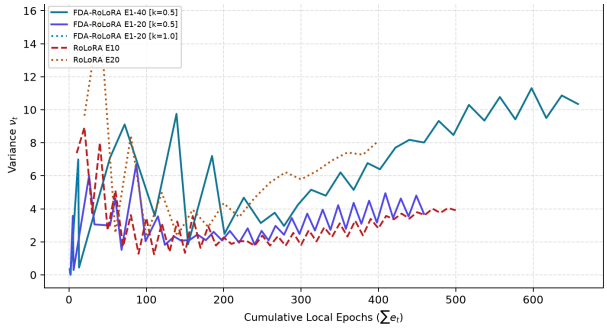

**(c)** Dynamic Local Epochs ($\bar{e}$)




**(d)** Factor Thresholds ($\theta_A, \theta_B$)




**(e)** Subspace Coordinate Variance




**(f)** Intra-Round Variance Drift

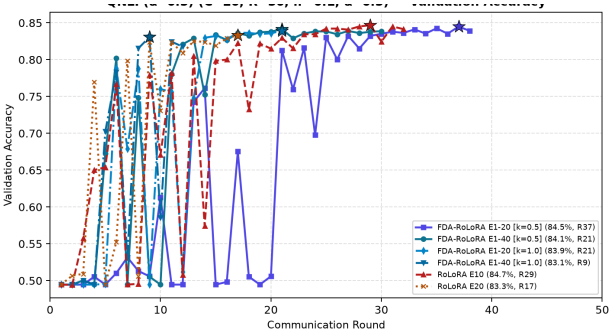
Figure 5.9: Multi-panel optimization telemetry on MNLI under Dirichlet label skew ($\alpha = 0.5, C = 20, R_{\max} = 50, \eta_c = 0.1$) comparing FDA-RoLoRA against static baselines.

**Question Answering (QNLI): Scalability & Suite-Leading Peak Accuracy.** Binary question-answering under skew demonstrates the scalability of closed-loop synchronization. As shown in Figure 5.10, FDA-RoLoRA ($k = 1.0$) attains the highest peak validation accuracy across the entire non-IID benchmark suite (**84.22**% on $C = 20$), outperforming all static baselines. In 50-client cohorts under acute skew, aggressive setpoint scaling ($k = 1.0$) caused optimization divergence, tripping early stopping at round 13.0 (152.3 MB) when validation accuracy degraded to 49.51%. In contrast, conservative midpoint scaling ($k = 0.5$) bounded client drift, attaining superior peak accuracy (**75.69**%) with exceptional inter-seed stability ($\pm$1.17%, compared to $\pm$7.18% for RoLoRA $E = 10$), demonstrating a 6.1$\times$ reduction in performance volatility under high client participation.

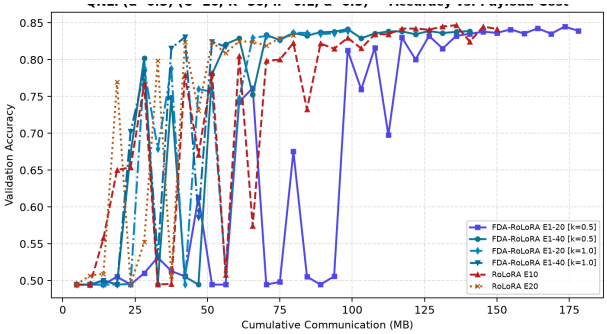
**Dynamic Round Synchronization and Realized Epoch Dynamics.** Figure 5.11 visualizes round-by-round realized local epoch allocations on MNLI and QNLI under statistical heterogeneity. When Dirichlet skew causes local client updates to diverge sharply in coordinate space, inter-client drift variance surges by approximately 4.0$\times$ (e.g., from 0.448 to 1.794 on QNLI). While static baselines execute a constant number of local passes regardless of parameter divergence, FDA-RoLoRA's closed-loop controller constrains local passes within $[e_{\min}, e_{\max}]$ in response to coordinate variance breaches, enforcing earlier server synchronization to mitigate client drift.

**Sentiment & Paraphrase: Cross-Task Drift and Anti-Collapse Protection.** Figure 5.12 provides a focused diagnostic synthesis on MNLI and QNLI of federated fine-tuning under acute Dirichlet label skew ($\alpha = 0.5$). The most pronounced empirical difference occurs on SST-2 with 50 clients. Under static $E = 10$, local over-fitting to skewed binary sentiment shards drives parameter trajectories into conflicting subspaces. Under static $E = 10$, RoLoRA remained trapped at the majority-class baseline ($\sim 50\%$) for the majority of rounds before an isolated surge to $59.00 \pm 7.86\%$ peak accuracy ($53.33 \pm 4.71\%$ final). In contrast, FDA-RoLoRA ($k = 1.0$) dynamically navigated the loss landscape to escape the plateau, attaining $\mathbf{85.95 \pm 3.96}\%$ peak accuracy ($\mathbf{81.36 \pm 4.09}\%$ final), representing a **+26.95% absolute improvement in peak accuracy**. On semantic paraphrase detection (QQP $C = 50$), FDA-RoLoRA triggers early stopping in 15.5 rounds (181.6 MB) versus RoLoRA's 22.5 rounds (263.7 MB), saving over 31.1% of total network bandwidth.

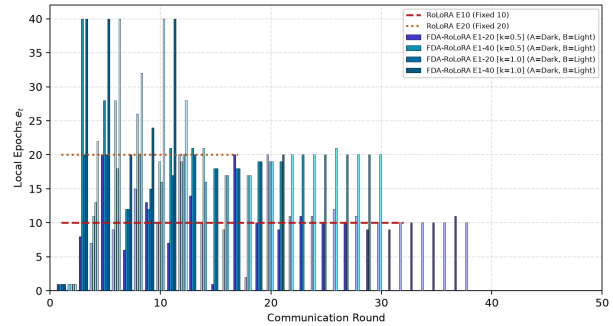
Across all four benchmarks, the client coordinate drift variance curves in Figure 5.12 reveal that parameter drift variance expands by approximately 4.0$\times$ compared to homogeneous environments. By dynamically modulating local epoch budgets in coordinate space, FDA-RoLoRA reliably bounds variance surges below destabilizing thresholds, preventing minority-class erasure and securing robust consensus.



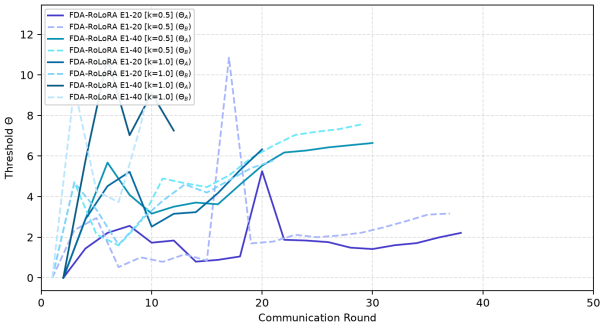

**(a)** Validation Accuracy



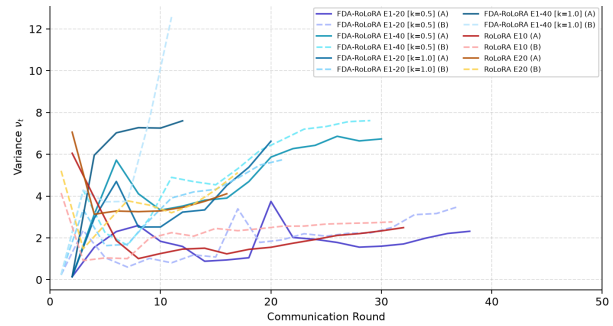

**(b)** Accuracy vs. Communication (MB)



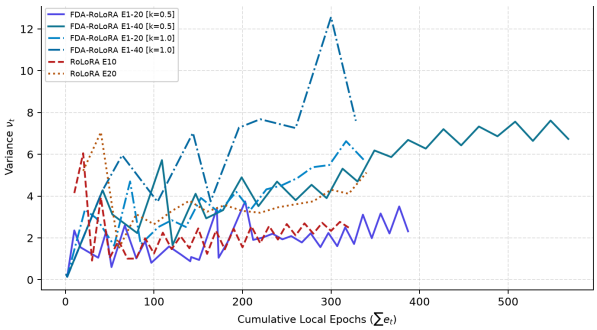

**(c)** Dynamic Local Epochs ($\bar{e}$)




**(d)** Factor Thresholds ($\theta_A, \theta_B$)




**(e)** Subspace Coordinate Variance




**(f)** Intra-Round Variance Drift

Figure 5.10: Multi-panel optimization telemetry on QNLI under Dirichlet label skew ($\alpha = 0.5, C = 20, R_{\max} = 50, \eta_c = 0.1$) comparing FDA-RoLoRA against static baselines.

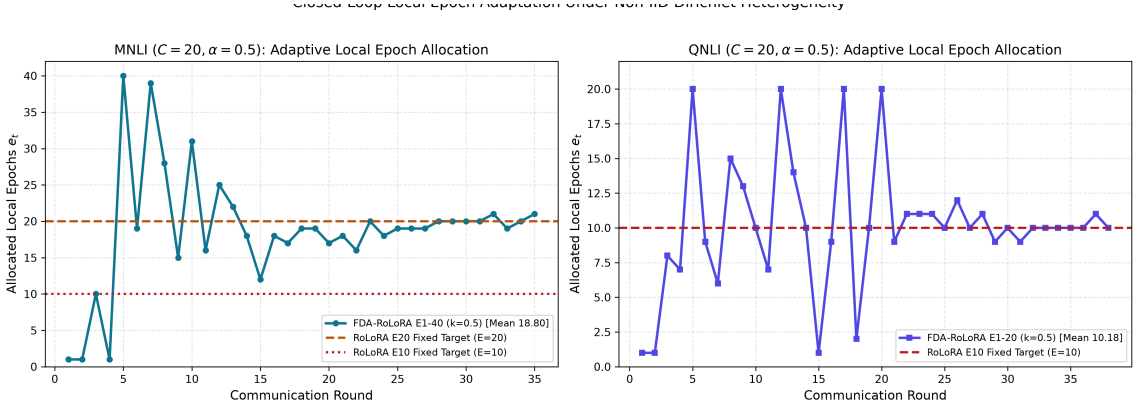



Figure 5.11: Empirical local epoch allocation dynamics under severe statistical heterogeneity (Dirichlet label skew $\alpha = 0.5$), illustrating the closed-loop setpoint tracking dynamics of FDA-RoLoRA on MNLI and QNLI under Dirichlet label skew ($\alpha = 0.5$).

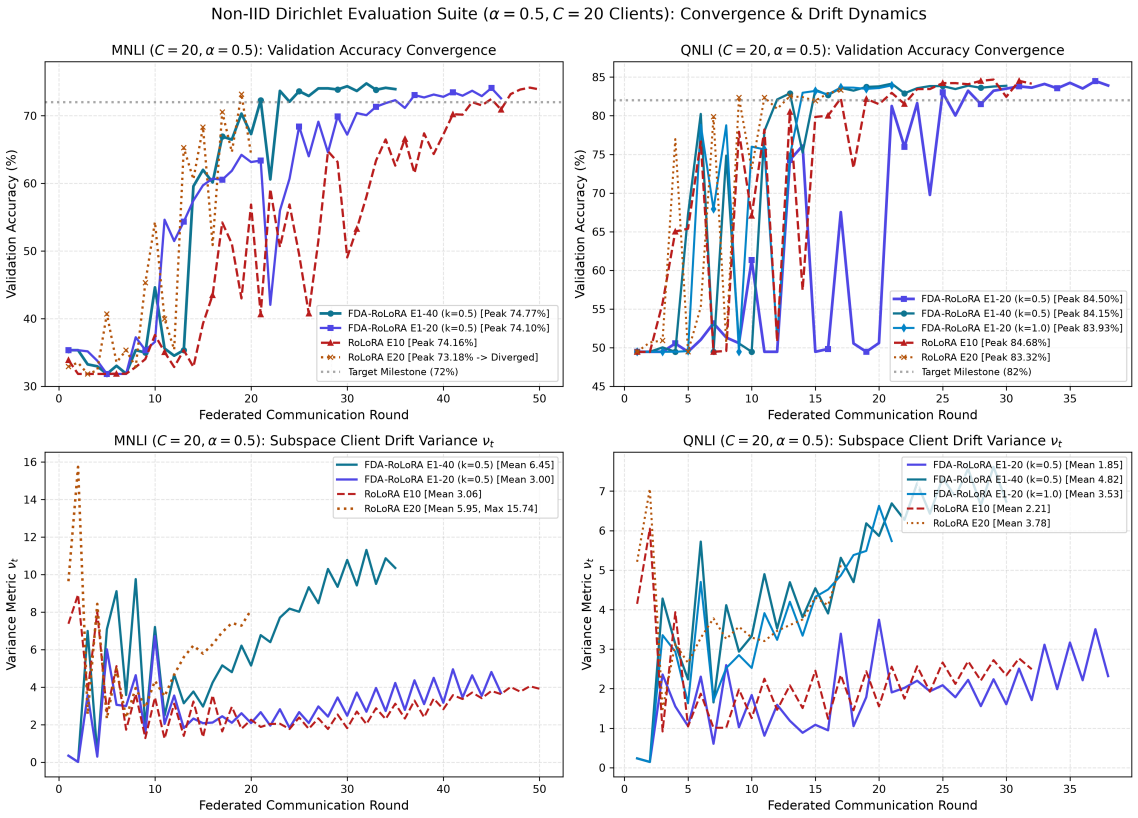



Figure 5.12: Convergence and client drift variance trajectories on MNLI and QNLI under severe Dirichlet statistical heterogeneity ($\alpha = 0.5$). Top panels present validation accuracy trajectories; bottom panels present client coordinate drift variance ($\sigma_t^2$).

# Chapter 6

# Conclusion and Future Work

This thesis investigated the integration of Federated Dynamic Averaging (FDA) with Parameter-Efficient Low-Rank Adaptation (LoRA) in federated learning. By formulating and evaluating FDA-RoLoRA, this work addressed both spatial communication overhead and rigid temporal synchronization schedules under data heterogeneity.

## 6.1 Synthesis of Results and Methodological Takeaways

The formulation, implementation, and empirical evaluation of FDA-RoLoRA provide several key takeaways:

1. **Preservation of Aggregation Linearity**: Dynamic averaging cannot be applied directly to standard federated LoRA. Updating both adapter matrices simultaneously creates a non-linear product aggregation error ($\frac{1}{N}\sum B_i A_i \neq \bar{B}\bar{A}$) that distorts client updates and misguides variance tracking. Alternating optimization updates only one factor ($A$ or $B$) per round, maintaining linear updates and allowing stable drift variance measurement.

2. **Direct Subspace Drift Monitoring**: Tracking drift variance on low-rank coordinates avoids the heavy memory and network overhead of full-model gradient estimation. Using separate thresholds $(\theta_A, \theta_B)$ for each factor allows the controller to adjust local epochs dynamically according to measured client divergence on the active matrix, preventing minority-class forgetting under skew.

3. **Empirical Foundation Model Benchmarks**: Evaluations using `roberta-base` across GLUE tasks show that dynamic synchronization can achieve substantial communication round savings while preserving or improving validation accuracy under non-IID client skew.

## 6.2 Limitations

The findings of this work should be considered in light of several practical and experimental limitations:

1. **Absence of Original RoLoRA Codebase**: The original implementation from the RoLoRA study [6] is not publicly available. Consequently, our work relied on an independent reimplementation from the paper's mathematical description, presenting significant engineering challenges in reconstructing and integrating its alternating dynamics into the FederatedScope-LLM framework.

2. **Omission of Critical Hyperparameters**: The RoLoRA paper did not report key experimental hyperparameters, most notably the task-specific optimal client learning rates. Substantial exploratory tuning was therefore required to identify stable, competitive learning rate schedules for the baseline before comparative benchmarks could be conducted.

3. **Hardware and Compute Constraints**: Limited local compute resources (2× NVIDIA A10 GPUs) restricted our experimental campaign to the 125M-parameter `roberta-base` model. While representative for masked language modeling and sequence classification, this prevented scaling to multi-billion parameter autoregressive Large Language Models (LLMs) and instruction-tuning workloads.

4. **Simulated Federated Environment**: All benchmarks were executed within a multi-process simulated federated environment on a single server. Real-world network factors, including physical latency, fluctuating client bandwidth, and intermittent client dropouts, were not modeled.

## 6.3 Future Work

Building upon this work, several concrete directions for future research emerge:

1. **Evaluation on Larger and More Diverse Models**: Expanding empirical evaluations beyond `roberta-base` to larger foundation models and modern autoregressive architectures (such as the LLaMA and Mistral families) to assess how dynamic coordinate variance scaling behaves on higher-capacity models and generative tasks.

2. **Integration with Other Federated LoRA Architectures**: Exploring how the core mechanics of FDA can be adapted to alternative federated parameter-efficient methods, such as SLoRA [3] (sparse coordinate exchanges), FFA-LoRA [27] (frozen down-projections), and standard simultaneous FedLoRA to test whether coordinate monitoring can mitigate client drift in wider architectural settings.

3. **Broader Experimental Scenarios for FDA-RoLoRA**: Conducting broader comparisons between FDA-RoLoRA and RoLoRA across varied federated conditions, including partial client participation per round, wider client cohort sizes ($C$), different client learning rates, and alternative matrix alternation orders (such as initializing with matrix $A$ first vs. matrix $B$ first).

4. **Framework Integration into FederatedScope-LLM**: Contributing FDA natively into the widely adopted FederatedScope-LLM framework [15] will make dynamic averaging readily accessible to the broader federated learning community and enable standardized benchmarking across emerging foundation model pipelines.

In conclusion, this work demonstrates that combining Federated Dynamic Averaging with alternating low-rank adaptation effectively addresses both parameter transmission volume and rigid synchronization schedules. By establishing a mathematically consistent substrate for coordinate variance monitoring, FDA-RoLoRA offers a practical, communication-efficient approach to federated foundation model adaptation.

# Bibliography


[1] Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, 2021.

[2] Noga Alon, Yossi Matias, and Mario Szegedy. The space complexity of approximating the frequency moments. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC)*, pages 20–29, 1996.

[3] Sara Babakniya, Souvik Ahmed, Yelong Shen, Bolin Ding, and Dawn Song. SLoRA: Federated Parameter-Efficient Fine-Tuning with Sparse Low-Rank Adaptation. *arXiv preprint arXiv:2311.12310*, 2023.

[4] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the Opportunities and Risks of Foundation Models. *arXiv preprint arXiv:2108.07258*, 2021.

[5] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, 2020.

[6] Shuangyi Chen, Yuanxin Guo, Yue Ju, Hardik Dalal, Zhongwen Zhu, and Ashish Khisti. Robust Federated Finetuning of LLMs via Alternating Optimization of LoRA. In *39th Conference on Neural Information Processing Systems (NeurIPS 2025)*, 2025.

[7] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient Finetuning of Quantized Large Language Models. In *Advances in Neural Information Processing Systems*, 2023.

[8] Georgios Frangias. Federated learning at TensorFlow using the geometric approach. Diploma thesis, Technical University of Crete, 2023.

[9] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-Efficient Transfer Learning for NLP. In *International Conference on Machine Learning*, 2019.

[10] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. Measuring the Effects of Non-Identical Data Distribution for Federated Visual Classification. *arXiv preprint arXiv:1909.06335*, 2019.

[11] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*, 2022.

[12] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. Advances and Open Problems in Federated Learning. *Foundations and Trends in Machine Learning*, 14(1–2):1–210, 2021.

[13] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh. SCAFFOLD: Stochastic Controlled Averaging for Federated Learning. In *International Conference on Machine Learning*, 2020.

[14] Diederik P Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*, 2015.

[15] Weirui Kuang, Bingyi Qian, Zitao Li, Daoyuan Chen, Dawei Gao, Xuchen Pan, Yuexiang Xie, Yaliang Li, Bolin Ding, and Jingren Zhou. FederatedScope-LLM: A Comprehensive Package for Federated Fine-Tuning of Large Language Models. *arXiv preprint arXiv:2309.00363*, 2023.

[16] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated Optimization in Heterogeneous Networks. In *Proceedings of Machine Learning and Systems*, 2020.

[17] Xiang Li, Kaixuan Huang, Wenhao Yang, Shusen Wang, and Zhihua Zhang. On the Convergence of FedAvg on Non-IID Data. In *International Conference on Learning Representations*, 2020.

[18] Xiang Lisa Li and Percy Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, 2021.

[19] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692*, 2019.

[20] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019.

[21] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Artificial Intelligence and Statistics*, pages 1273–1282. PMLR, 2017.

[22] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.

[23] Sashank Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečnỳ, Sanjiv Kumar, and H Brendan McMahan. Adaptive Federated Optimization. In *International Conference on Learning Representations*, 2021.

[24] Herbert Robbins and Sutton Monro. A Stochastic Approximation Method. *The Annals of Mathematical Statistics*, pages 400–407, 1951.

[25] Izchak Sharfman, Assaf Schuster, and Daniel Keren. A geometric approach to monitoring threshold functions over distributed data streams. In *Proceedings of the 2006 ACM SIGMOD international conference on Management of data*, pages 301–312, 2006.

[26] Omer Subasi, Oceane Bel, Joseph Manzano, and Kevin Barker. The landscape of modern machine learning: A review of machine, distributed and federated learning. *arXiv preprint arXiv:2312.03120*, 2023.

[27] Youbang Sun, Zitao Li, Yaliang Li, and Bolin Ding. Improving LoRA in Privacy-preserving Federated Learning. In *The Twelfth International Conference on Learning Representations*, 2025.

[28] Michail Theologitis, Vasilis Samoladas, and Antonios Deligiannakis. FDA-Opt: Communication-Efficient Federated Fine-Tuning of Language Models. *arXiv preprint arXiv:2505.04535*, 2025. Code available at `https://github.com/michaeltheologitis/FDA-Opt`.

[29] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*, 2023.

[30] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, 2017.

[31] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. In *International Conference on Learning Representations*, 2019.

[32] Ziyao Wang, Zheyu Shen, Yexiao He, Guoheng Sun, Hongyi Wang, Lingjuan Lyu, and Ang Li. FLoRA: Federated Fine-Tuning Large Language Models with Heterogeneous Low-Rank Adaptations. In *NeurIPS*, 2025.

[33] Yiyuan Yang, Guodong Long, Qinghua Lu, Liming Zhu, Jing Jiang, and Chengqi Zhang. Federated Low-Rank Adaptation for Foundation Models: A Survey. *arXiv preprint arXiv:2505.13502*, 2025.

[34] Mikhail Yurochkin, Mayank Agarwal, Soumya Banerjee, Rebecca Chen, Dennis Sun, and Yasaman Khazaeni. Bayesian Nonparametric Federated Learning of Neural Networks. In *International Conference on Machine Learning (ICML)*, pages 7252–7261, 2019.

[35] Jianyi Zhang, Saeed Vahidian, Martin Kuo, Chunyuan Li, Ruiyi Zhang, Tong Yu, Guoyin Wang, and Yiran Chen. Towards Building the Federated GPT: Federated Instruction Tuning. *arXiv preprint arXiv:2305.05644*, 2023.

[36] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. In *International Conference on Learning Representations*, 2023.

[37] Zhuangdi Zhu, Junlin Hong, and Shiyu Ji. Federated Learning on Non-IID Data: A Survey. *Neurocomputing*, 465:371–390, 2021.